

CleanEDA

March 30, 2025

1 Personal Information

Name: **Ruben Curiël**

StudentID: **12727431**

Email: rzff.curiel@gmail.com

Submitted on: **23.3.2025**

Github: <https://github.com/rzff/MsCThesisRubenCuriel2024/tree/main/Code>

2 Data Context

In this section you should introduce the datasources and datasets which you will be working with. Explain where they are from as well as their domain. Give an overview of what the context of the data is. You should not spend more than 1 to 2 paragraphs here as the core information will be in the next section.

CBS - Labor participation - GDP Composition - Housing - Energy Prices

World Bank - Commodity Price Data (The Pink Sheet)

KNMI - Daily temperature - precipitation - wind speed **Copernicus Climate Datastore** - SIC
- SC - SST - Nao Variablity

ENTSOE - Load Consumption

```
[94]: import pandas as pd
import re
from ydata_profiling import ProfileReport
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import scipy.stats as stats
from scipy.stats import shapiro, pearsonr # Keep this if needed
import os
import netCDF4 as nc
import pandas as pd
import datetime
from sklearn.decomposition import PCA
```

```

import numpy as np
import datetime
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from prophet import Prophet
import warnings
import math
from math import sqrt
import time
warnings.filterwarnings("ignore")

def Create_Hist_Normal_Plot_With_Stats_Outliers(df):
    # Select numeric columns only
    numeric_cols = df.select_dtypes(include=['number']).columns
    num_cols = len(numeric_cols)

    if num_cols == 0:
        print("No numeric columns to plot.")
        return

    # Create subplots in a single column layout
    fig, axes = plt.subplots(num_cols, 1, figsize=(8, num_cols * 4))

    # If there's only one plot, put it in a list for consistency.
    if num_cols == 1:
        axes = [axes]

    for i, col in enumerate(numeric_cols):
        # Convert column values to numeric and drop non-convertible values
        data = pd.to_numeric(df[col], errors='coerce').dropna()
        if data.empty:
            axes[i].text(0.5, 0.5, f'No valid numeric data in {col}',
↪ha='center')
            axes[i].set_title(f'Histogram for {col}')
            continue

        # Compute basic statistics
        mean = data.mean()
        median = data.median()

```

```

std = data.std()

# Identify outliers using the IQR method
Q1 = data.quantile(0.25)
Q3 = data.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = data[(data < lower_bound) | (data > upper_bound)]
outlier_count = len(outliers)

# Plot the histogram (density normalized)
axes[i].hist(data, bins=30, density=True, alpha=0.6, color='gray',
↪edgecolor='black')

# Plot the normal curve based on the computed mean and std
xmin, xmax = data.min(), data.max()
x = np.linspace(xmin, xmax, 100)
p = stats.norm.pdf(x, mean, std)
axes[i].plot(x, p, 'b-', linewidth=2, label='Normal Curve')

# Plot vertical dashed lines for mean and median
axes[i].axvline(mean, color='red', linestyle='dashed', linewidth=1.5,
↪label='Mean')
axes[i].axvline(median, color='orange', linestyle='dashed', linewidth=1.
↪5, label='Median')

# Perform Shapiro-Wilk test if there are enough data points (at least 3)
if len(data) >= 3:
    shapiro_stat, shapiro_p = stats.shapiro(data)
    shapiro_text = f"Shapiro: stat={shapiro_stat:.2f}, p={shapiro_p:.
↪4f}"
else:
    shapiro_text = "Shapiro: Not enough data"

# Create annotation text that includes the statistics and outlier count
stats_text = f"Mean: {mean:.2f}, Median: {median:.2f}, Std: {std:.2f}"
outlier_text = f"Outliers: {outlier_count}"
annotation_text = f"{stats_text}\n{outlier_text}\n{shapiro_text}"

# Annotate below the plot with extra vertical padding (adjust
↪y-coordinate as needed)
axes[i].text(0.5, -0.30, annotation_text, transform=axes[i].transAxes,
             ha='center', va='center', fontsize=9,
             bbox=dict(facecolor='white', alpha=0.5, edgecolor='none'))

# Set the title and legend

```

```

axes[i].set_title(f'Histogram of {col}')
axes[i].legend()

# Rotate tick labels for readability
for label in axes[i].get_xticklabels():
    label.set_rotation(45)
for label in axes[i].get_yticklabels():
    label.set_rotation(45)

# Adjust layout to give room for annotations and prevent label overlap
fig.tight_layout(pad=3.0)
plt.subplots_adjust(hspace=1.2)
plt.show()

def Create_QQ_Plot(df):
    # Select numeric columns only
    numeric_cols = df.select_dtypes(include=['number']).columns
    num_cols = len(numeric_cols)

    if num_cols == 0:
        print("No numeric columns to plot.")
        return

    # Create subplots in a single column layout
    fig, axes = plt.subplots(num_cols, 1, figsize=(8, num_cols * 4))

    if num_cols == 1:
        axes = [axes]

    # Loop through each numeric column and create a QQ-plot
    for i, col in enumerate(numeric_cols):
        data = pd.to_numeric(df[col], errors='coerce').dropna()
        if data.empty:
            axes[i].text(0.5, 0.5, f'No valid numeric data in {col}',
                ↪ha='center')
            axes[i].set_title(f'QQ Plot for {col}')
            continue

        # Generate the QQ-plot using statsmodels
        sm.qqplot(data, line='s', ax=axes[i])
        axes[i].set_title(f'QQ Plot of {col}')

        # Rotate tick labels for readability
        for label in axes[i].get_xticklabels():
            label.set_rotation(45)
        for label in axes[i].get_yticklabels():
            label.set_rotation(45)

```

```

# Adjust layout so that the plots are spaced out properly
fig.tight_layout(pad=3.0)
plt.subplots_adjust(hspace=0.8)
plt.show()

def convert_to_numeric(df):
    for col in df.select_dtypes(include=['object']).columns:
        # Replace commas with periods and convert to numeric (non-convertible
        ↪ values become NaN)
        df[col] = df[col].str.replace(',', '.')
        df[col] = pd.to_numeric(df[col], errors='coerce')
    return df

def extract_int(x):
    digits = re.sub(r'\D', '', x)
    return int(digits) if digits else 0

def convert_quarter_string(s):
    s = s.strip()
    # Expected pattern: "YYYY <number>[optional letter] kwartaal"
    pattern = r"^(?d{4})\s+(?d+)[a-zA-Z]*\s+kwartaal$"
    m = re.fullmatch(pattern, s)
    if m:
        year = m.group(1)
        quarter = m.group(2)
        return pd.Period(f"{year}Q{quarter}", freq='Q')
    else:
        return None

def convert_year_string(s):
    s = s.strip()
    # Expected: exactly a four-digit year.
    m = re.fullmatch(r"\d{4}", s)
    if m:
        return int(s)
    else:
        return None

def convert_year_month_string(s):
    # If the value is already a Period, convert it directly.
    if isinstance(s, pd.Period):
        return s.to_timestamp(how='end')

    s = str(s).strip().lower() # Normalize the string (e.g., lowercase for
    ↪ Dutch month names)

```

```

# Mapping for Dutch month names to month numbers
dutch_months = {
    'januari': '01',
    'februari': '02',
    'maart': '03',
    'april': '04',
    'mei': '05',
    'juni': '06',
    'juli': '07',
    'augustus': '08',
    'september': '09',
    'oktober': '10',
    'november': '11',
    'december': '12'
}

# First, try the direct conversion to a Period with monthly frequency.
try:
    period_obj = pd.Period(s, freq='M')
    return period_obj
except Exception:
    pass

# New branch: try to match format like "1960M02"
m = re.match(r"(\d{4})m(\d{1,2})", s)
if m:
    year = int(m.group(1))
    month = int(m.group(2))
    period_obj = pd.Period(f"{year}--{month:02d}", freq='M')
    return period_obj

# Try to match numeric year-month (formats like "2025-01" or "2025 1")
m = re.match(r"(\d{4})[-\s]?(\d{1,2})", s)
if m:
    year = int(m.group(1))
    month = int(m.group(2))
    period_obj = pd.Period(f"{year}--{month:02d}", freq='M')
    return period_obj

# Try to match Dutch month names (formats like "2025 januari")
m = re.match(r"(\d{4})\s*([a-zA-Z]+)", s)
if m:
    year = int(m.group(1))
    month_str = m.group(2)
    month_num = dutch_months.get(month_str.lower())
    if month_num:
        period_obj = pd.Period(f"{year}--{month_num}", freq='M')

```

```

        return period_obj

    # If none of the above worked, return None.
    return None

def expand_quarter_to_daily(df, quarter_col=None, use_index=False):

    expanded_rows = []

    for idx, row in df.iterrows():
        # Extract the quarter string either from the index or the specified
        ↪column.
        if use_index:
            quarter_str = str(idx).strip()
        else:
            if quarter_col is None:
                raise ValueError("quarter_col must be provided if use_index is
                ↪False.")
            quarter_str = str(row[quarter_col]).strip()

        # Skip rows that should be dropped.
        if quarter_str == "Bron: CBS":
            continue

        period = convert_quarter_string(quarter_str)
        if period is None:
            continue # Skip rows not matching the expected quarter format.

        start_date = period.start_time
        end_date = period.end_time

        # Create a daily date range for the quarter.
        daily_index = pd.date_range(start=start_date, end=end_date, freq='D')
        # Duplicate the row for each day.
        row_expanded = pd.DataFrame([row] * len(daily_index), index=daily_index)
        row_expanded['date'] = daily_index

        expanded_rows.append(row_expanded)

    return pd.concat(expanded_rows) if expanded_rows else pd.DataFrame()

def expand_year_to_daily(df, year_col=None, use_index=False):
    expanded_rows = []

```

```

    for idx, row in df.iterrows():
        # Extract the year string either from the index or from the specified
        ↪column.
        if use_index:
            year_str = str(idx).strip()
        else:
            if year_col is None:
                raise ValueError("year_col must be provided if use_index is
                ↪False.")
            year_str = str(row[year_col]).strip()

        year_int = convert_year_string(year_str)
        if year_int is None:
            continue # Skip rows that do not have a valid year.

        start_date = pd.Timestamp(year=year_int, month=1, day=1)
        end_date = pd.Timestamp(year=year_int, month=12, day=31)

        # Create a daily date range for the year.
        daily_index = pd.date_range(start=start_date, end=end_date, freq='D')
        # Duplicate the row for each day.
        # Convert the row to a dictionary and then create a DataFrame by replicating
        ↪the dictionary values.
        row_dict = row.to_dict()
        row_expanded = pd.DataFrame({key: [value] * len(daily_index) for key,
        ↪value in row_dict.items()}),
                                   index=daily_index)
        row_expanded['date'] = daily_index

        expanded_rows.append(row_expanded)

    return pd.concat(expanded_rows) if expanded_rows else pd.DataFrame()

def expand_halfjaar_to_months(period):
    """
    Expands a half-year period string into a list of monthly strings in
    ↪'YYYY-MM' format.
    """
    parts = period.split()
    if len(parts) < 2:
        raise ValueError("Invalid period format. Expected 'YYYY Xe ...'.")

    year_str, half_indicator = parts[0], parts[1]
    year = int(year_str)

    if half_indicator.startswith('1e'):
        start_month, end_month = 1, 6

```



```

elif half_indicator.startswith('2e'):
    start_month, end_month = 7, 12
else:
    raise ValueError(f"Unrecognized half identifier: '{half_indicator}'.")

return [f"{year}-{month:02d}" for month in range(start_month, end_month + 1)]

def expand_year_month_to_daily(df, year_month_col=None, use_index=False):
    expanded_rows = []

    for idx, row in df.iterrows():
        # Extract the year-month string either from index or a specified column.
        if use_index:
            year_month_str = str(idx).strip()
        else:
            if year_month_col is None:
                raise ValueError("year_month_col must be provided if use_index_
is False")
            year_month_str = str(row[year_month_col]).strip()

        period = convert_year_month_string(year_month_str)
        if period is None:
            continue # Skip rows that don't have a valid year-month

        # Determine start and end dates for the month.
        start_date = period.start_time # first day of the month
        end_date = period.end_time # last day of the month

        # Create a daily date range for that month.
        daily_index = pd.date_range(start=start_date, end=end_date, freq='D')

        # Duplicate the row for each day.
        # Instead of repeating the row (which can cause duplicate-index issues),
        # we create a dictionary with each column's value repeated.
        row_dict = row.to_dict()
        daily_df = pd.DataFrame({key: [value] * len(daily_index) for key, value_
in row_dict.items()}),
                                index=daily_index)

        # Optionally add a 'date' column.
        daily_df['date'] = daily_index

        expanded_rows.append(daily_df)

    if expanded_rows:
        return pd.concat(expanded_rows)
    else:

```

```

        return pd.DataFrame()

import pandas as pd

def read_data(path: str) -> pd.DataFrame:
    """
    Read the data.
    Strip spaces from the columns.
    Rename all the columns (abbreviations -> full names).
    """

    weather = pd.read_csv(path)

    weather.columns = weather.columns.str.strip()
    weather = weather.applymap(lambda x: x.strip() if isinstance(x, str) else x)
    weather.replace('', None, inplace=True)

    column_mapping = {
        'YYYYMMDD': 'Date',
        'RH': 'DailyPrecipitation',
        'RHX': 'MaxHourlyPrecipitation',
        'RHXH': 'HDMaxPrecipitation',
        'TG': 'DailyMeanTemperature',
        'TN': 'HourlyMinTemperature',
        'TNH': 'HDMINTemperature',
        'TX': 'HourlyMaxTemperature',
        'TXH': 'HDMaxTemperature',
        'FG': 'DailyMeanWindspeed',
        'FHX': 'MaxHourlyMeanWindspeed',
        'FHXH': 'HDMaxMeanWindspeed',
        'FHN': 'MinHourlyMeanWindspeed',
        'FHNH': 'HDMINMeanWindspeed'
    }

    weather = weather[column_mapping.keys()]
    weather.rename(columns=column_mapping, inplace=True)

    return weather

def filter_on_year(weather: pd.DataFrame) -> pd.DataFrame:
    """
    Filter on only 2021 and later and reset index after filtering.
    """

    weather = weather[weather['Date'] >= 20210101]
    weather.reset_index(drop=True, inplace=True)

```

```

return weather

def cast_datekey(weather: pd.DataFrame) -> pd.DataFrame:
    """
    Cast datekey 20210101 to format 2021/01/01
    """
    weather['Date'] = pd.to_datetime(weather['Date'], format='%Y%m%d').dt.
    ↪strftime('%Y/%m/%d')

    return weather

def cast_hour_columns(weather: pd.DataFrame) -> pd.DataFrame:
    """
    Cast hour columns to integer.
    """
    hour_cols = [
        'HDMaxPrecipitation',
        'HDMinTemperature',
        'HDMaxTemperature',
        'HDMaxMeanWindspeed',
        'HDMinMeanWindspeed'
    ]

    for i in hour_cols:
        weather[i] = weather[i].astype(int)

    return weather

def convert_columns(weather: pd.DataFrame) -> pd.DataFrame:
    """
    Divide the following columns by 10 to get actual number, since they are 0.1_
    ↪of the actual number.
    """
    cols_to_convert = [
        'DailyPrecipitation',
        'MaxHourlyPrecipitation',
        'DailyMeanTemperature',
        'HourlyMaxTemperature',
        'DailyMeanWindspeed',
        'MinHourlyMeanWindspeed',
        'MaxHourlyMeanWindspeed'
    ]

    for i in cols_to_convert:
        weather[i] = weather[i] / 10

```

```

    return weather

def create_date_columns(weather: pd.DataFrame) -> pd.DataFrame:
    """
    Cast date column (2021-01-01 instead of 2021/01/01) and add month and year_
    ↪ columns
    Create a week key combining year and week number
    """
    weather['Date'] = pd.to_datetime(weather['Date'], format='%Y/%m/%d')
    weather['Month'] = weather['Date'].dt.month
    weather['Year'] = weather['Date'].dt.year
    weather['Week'] = weather['Date'].dt.isocalendar().week

    weather['WeekKey'] = weather['Year'].astype(str) + weather['Week'].
    ↪ apply(lambda x: '{:02d}'.format(x))
    weather['day_of_week'] = weather['Date'].dt.day_name()

    return weather

def preprocess_weather(path: str) -> pd.DataFrame:
    """
    Preprocess the weather data.
    """
    weather = read_data(path)
    weather = filter_on_year(weather)
    weather = cast_datekey(weather)
    weather = cast_hour_columns(weather)
    weather = convert_columns(weather)
    weather = create_date_columns(weather)

    return weather

import pandas as pd

def preprocess_nao(path: str) -> pd.DataFrame:
    """
    Reset index, index is now the year and we want this as a column.
    Melt the dataset into a year, month and NAO column.
    Rename index to Year.
    Map the months (now strings) to corresponding month numbers.
    """
    nao = pd.read_csv(path, delim_whitespace=True, index_col=0)

```

```

    nao.reset_index(inplace=True)
    nao_melted = pd.melt(nao, id_vars=['index'], var_name='Month',
↳value_name='NAO')
    nao_melted.rename(columns={'index': 'Year'}, inplace=True)
    month_mapping = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun': 6,
        'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11, 'Dec': 12}
    nao_melted['Month'] = nao_melted['Month'].map(month_mapping)

    return nao_melted

import pandas as pd

def add_datetime_columns(df):
    # Create a copy to avoid modifying the original DataFrame
    df = df.copy()

    # Initialize datetime_index
    datetime_index = None

    # Check if the index is a PeriodIndex
    if isinstance(df.index, pd.PeriodIndex):
        datetime_index = df.index.to_timestamp()
    else:
        # Check if the index contains Period objects (for non-empty index)
        if df.index.size > 0 and isinstance(df.index[0], pd.Period):
            datetime_index = pd.DatetimeIndex([p.to_timestamp() for p in df.
↳index])
        else:
            # Convert the index to datetime using pandas to_datetime
            datetime_index = pd.to_datetime(df.index)

    # Extract each component and assign to new columns
    df['Year'] = datetime_index.year
    df['Month'] = datetime_index.month
    df['Day'] = datetime_index.day
    df['Hour'] = datetime_index.hour
    df['Minute'] = datetime_index.minute

    for col in ['Year', 'Month', 'Day', 'Hour', 'Minute']:
        df[col] = df[col].astype('Int64')
    return df

def check_missing_rows(df: pd.DataFrame) -> pd.DataFrame:
    """
    Checks for rows with NaN, empty strings, or whitespace-only strings.

```

```

Returns a DataFrame with the original index and a 'has_missing' boolean
↳ column.
"""
# Check for NaN/None
na_mask = df.isnull()

# Check for empty strings
empty_str_mask = (df == '')

# Check for whitespace-only strings (SAFELY, without .strip() on
↳ non-strings)
whitespace_mask = df.applymap(lambda x:
    isinstance(x, str) and (x.strip() == '') # <-- Check if string FIRST
)

# Combine masks
combined_mask = na_mask | empty_str_mask | whitespace_mask

# Check row-wise
has_missing = combined_mask.any(axis=1)

# Return with column name
return has_missing.to_frame(name='has_missing')

def get_rows_with_missing(df: pd.DataFrame) -> pd.DataFrame:
    """
    Returns rows from the original DataFrame where any missing values are
    ↳ detected.
    """
    has_missing = check_missing_rows(df)['has_missing']
    return df[has_missing]

def Create_EDA(df):
    # Check for missing values
    missing_report = check_missing_rows(df)

    # Get rows with missing values (for potential inspection)
    df_missing_rows = df[missing_report['has_missing']]

    # Print statistics
    print(f'Amount of days with missing values: {df.isna().sum() / len(df)}')
    print('\nDescriptives of labor participation:\n')
    print(df.describe())
    print('\n' + '-'*50 + '\n')

    # Generate plots
    print('Distribution and QQ plots:\n')

```

```

QQ_plots = Create_QQ_Plot(df)
Distribution_Plots = Create_Hist_Normal_Plot_With_Stats_Outliers(df)

# Display plots (if not automatically rendered)
plt.show()

def merge_on_common_time_columns(df1, df2):
    # Define the hierarchy of possible merge keys
    try:
        df2 = convert_to_numeric(df2)
    except:
        pass
    possible_keys = ["Year", "Month", "Day", "Hour", "Minute"]

    # Find common columns present in both DataFrames
    common_keys = [key for key in possible_keys if key in df1.columns and key
↪in df2.columns]
    if not common_keys:
        raise ValueError("No common time columns (Year/Month/Day) to merge on.")

    # Merge using the highest available time granularity
    merged_df = pd.merge(df1, df2, on=common_keys, how="inner")
    return merged_df

def calculate_correlations_with_pvalues(CompleteDataset,
↪target_column='LoadConsumption'):
    """
    Calculate Pearson correlations with p-values for a target column against
↪all other columns.

    Args:
        CompleteDataset (pd.DataFrame): DataFrame containing all variables
        target_column (str): Column name to calculate correlations against

    Returns:
        pd.DataFrame: Sorted correlations with p-values
    """
    # Calculate correlations and p-values
    correlations = []
    pvalues = []

    for col in CompleteDataset.columns:
        if col != target_column:
            # Drop NA pairs for valid correlation calculation
            clean_data = CompleteDataset[[target_column, col]].dropna()
            if len(clean_data) < 2:
                corr = np.nan

```

```

        pval = np.nan
    else:
        corr, pval = pearsonr(clean_data[target_column],
↪clean_data[col])
        correlations.append(corr)
        pvalues.append(pval)
    else:
        correlations.append(1.0) # Correlation with self
        pvalues.append(0.0)      # P-value with self

# Create DataFrame with results
result_df = pd.DataFrame({
    'Variable': CompleteDataset.columns,
    'Correlation': correlations,
    'P_Value': pvalues
}).set_index('Variable')

# Sort by absolute correlation value descending
sorted_df = result_df.reindex(
    result_df['Correlation'].abs().sort_values(ascending=False).index
)

return sorted_df

```

3 Preprocessing

Since I will be using a lot of datasets that need to be combined I needed to make all the datasets as uniform as possible. This is done by 1. Checking all the datasets on completeness, since most of the data is from the CBS is fairly clean already - To do so I used descriptives, QQ-plots and distributions to find anomalies 2. Create datetime values to eventually merge on. Since the Loadconsumption used a column for Year, Month Day Hour and Minute I recreated this for the other datasets as well

4 Analysis 1 - Labor participation

```

[2]: # Non percentage values are /1000 of real value.
LaborParticipation = pd.read_csv('/Users/ruben/Documents/GitHub/
↪MsCThesisRubenCuriel2024/Data/CBS/
↪Arbeidsdeelname__kerncijfers_seizoengecorrigeerd_21022025_095354.csv',
sep=';', header=3)

LaborParticipation = LaborParticipation.drop([0])

# #generating a report
def LaborParticipationPreprocess(LaborParticipation):

```



```

    LaborParticipation['Unnamed: 0'] = LaborParticipation['Unnamed: 0'].
↪apply(lambda x: convert_quarter_string(x))
    LaborParticipation['Unnamed: 0'] = (
        pd.PeriodIndex(LaborParticipation['Unnamed: 0'], freq="Q")
        .to_timestamp() # Converts Period to datetime
    )
    LaborParticipation.set_index('Unnamed: 0', inplace=True)
    LaborParticipation.index.names = ['Date']
    # In the data full years could be skipped, due to the conversion these
↪became null thus we can drop them
    LaborParticipation = LaborParticipation[LaborParticipation.index.notnull()]
    LaborParticipation['Beroepsbevolking|Werkloosheidspercentage'] =
↪LaborParticipation['Beroepsbevolking|Werkloosheidspercentage'].astype(str).
↪str.replace(',', '.')
    LaborParticipation = LaborParticipation.apply(pd.to_numeric,
↪errors='coerce')
    return LaborParticipation

df_LaborParticipation = LaborParticipationPreprocess(LaborParticipation)

Create_EDA(df_LaborParticipation)

```

Amount of days with missing values: Beroepsbevolking|werkloze

beroepsbevolking Werkloze	beroepsbevolking	0.0
Beroepsbevolking werkloze	beroepsbevolking Kortdurig	0.0
Beroepsbevolking werkloze	beroepsbevolking Langdurig of onbekend	0.0
Beroepsbevolking Werkloosheidspercentage		0.0

dtype: float64

Descriptives of labor participation:

	Beroepsbevolking werkloze	beroepsbevolking Werkloze	beroepsbevolking \
count	48.000000		
mean	522.354167		
std	155.867441		
min	327.000000		
25%	374.000000		
50%	461.000000		
75%	689.000000		
max	793.000000		

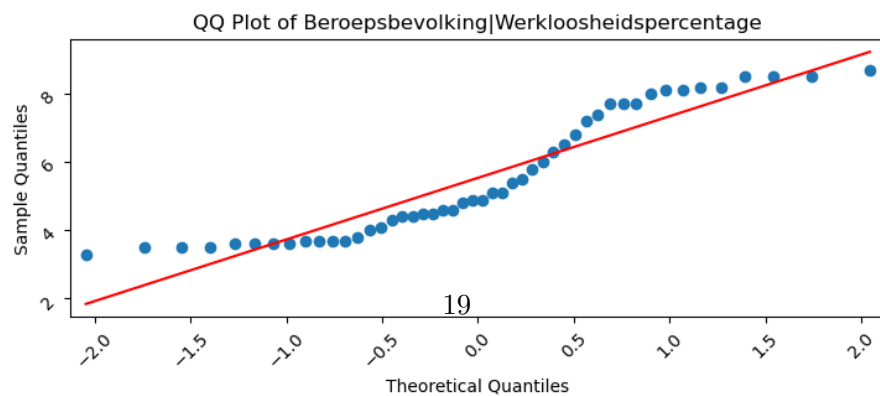
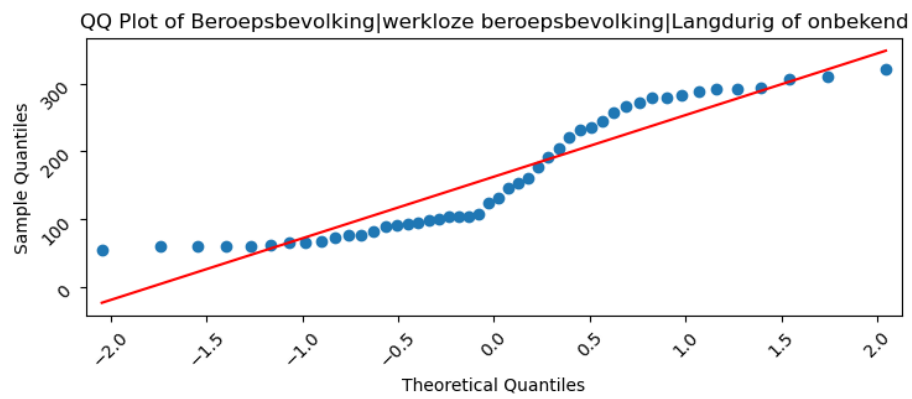
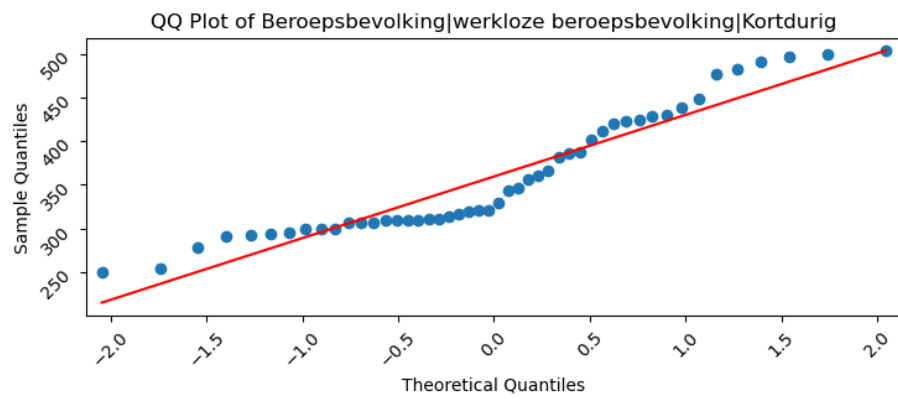
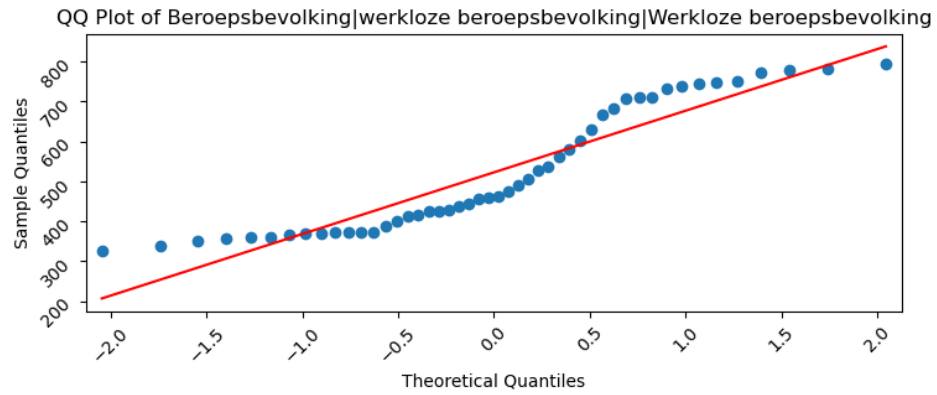
	Beroepsbevolking werkloze	beroepsbevolking Kortdurig	\
count	48.000000		
mean	359.458333		
std	71.239768		
min	250.000000		

25%	306.750000
50%	325.000000
75%	420.750000
max	503.000000

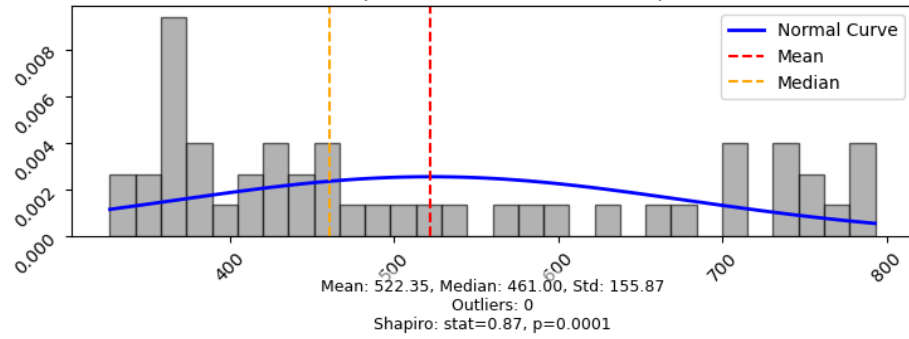
	Beroepsbevolking werkloze beroepsbevolking Langdurig of onbekend \
count	48.000000
mean	162.937500
std	91.469657
min	55.000000
25%	81.500000
50%	128.000000
75%	260.000000
max	321.000000

	Beroepsbevolking Werkloosheidspercentage
count	48.000000
mean	5.537500
std	1.825193
min	3.300000
25%	3.775000
50%	4.900000
75%	7.475000
max	8.700000

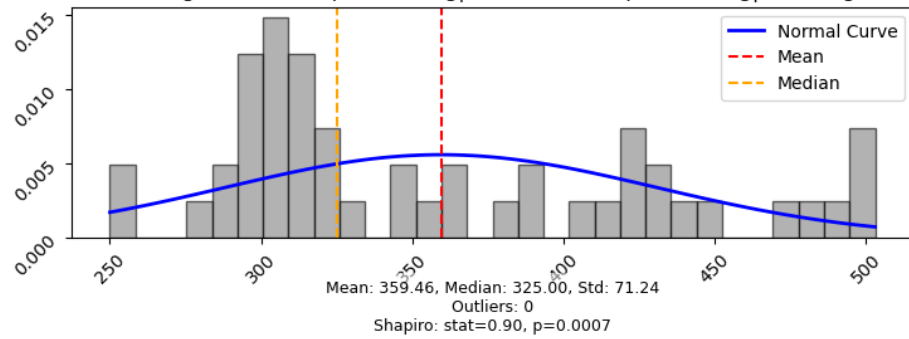
Distribution and QQ plots:



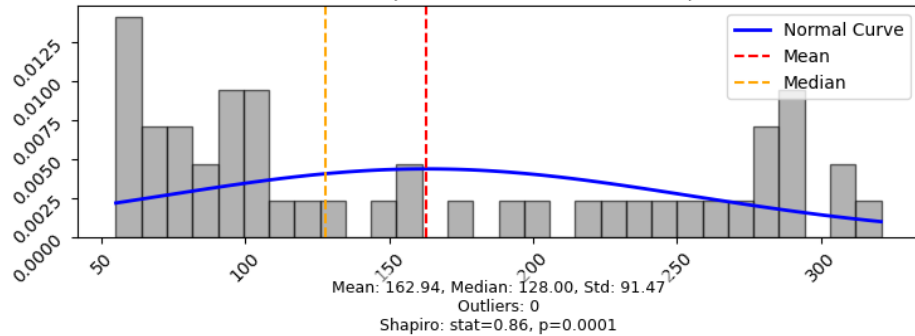
Histogram of Beroepsbevolking|werkloze beroepsbevolking|Werkloze beroepsbevolking



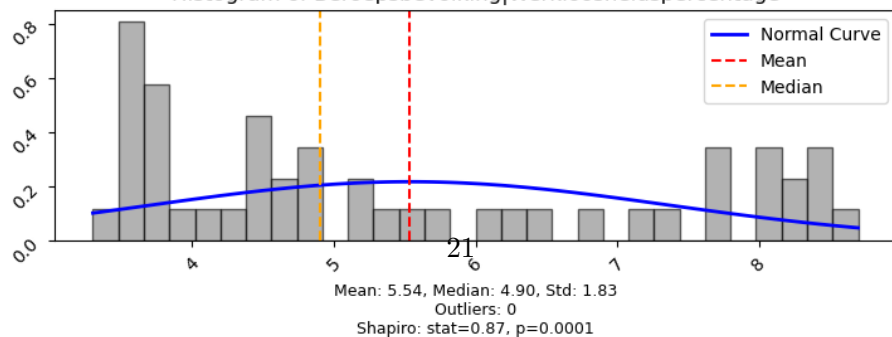
Histogram of Beroepsbevolking|werkloze beroepsbevolking|Kortdurig



Histogram of Beroepsbevolking|werkloze beroepsbevolking|Langdurig of onbekend



Histogram of Beroepsbevolking|Werkloosheidspercentage



5 Analysis 2 - Consumer Price Index

Nan Values can be explained due to the fact Jaarmutatie was only calculated from 1997 onwards, therefore for the eventual data set the start date will be 1997-1-1

```
[3]: ConsumerPriceIndex = pd.read_csv('/Users/ruben/Documents/GitHub/
↳MsCThesisRubenCuriel2024/Data/CBS/
↳Consumentenprijzen__prijsindex_2015_100_01032025_140302.csv',sep=';',
↳header=3)
ConsumerPriceIndex.drop([0])

def ConsumerPriceIndexPreprocess(ConsumerPriceIndex):
    ConsumerPriceIndex.columns += ' | ' + ConsumerPriceIndex.iloc[0]
    ConsumerPriceIndex = ConsumerPriceIndex.drop(0)

    ConsumerPriceIndex['Unnamed: 0 | Perioden'] = ConsumerPriceIndex['Unnamed: 0 | Perioden'].apply(lambda x: convert_year_month_string(x))
    ConsumerPriceIndex.set_index(['Unnamed: 0 | Perioden'], inplace=True)
    ConsumerPriceIndex.index.names = ['Date']
    ConsumerPriceIndex = convert_to_numeric(ConsumerPriceIndex)
    return ConsumerPriceIndex

# ConsumerPriceIndexPreprocess(ConsumerPriceIndex)
df_ConsumerPriceIndex = ConsumerPriceIndexPreprocess(ConsumerPriceIndex)
df_ConsumerPriceIndex.index.to_timestamp

Create_EDA(df_ConsumerPriceIndex)
```

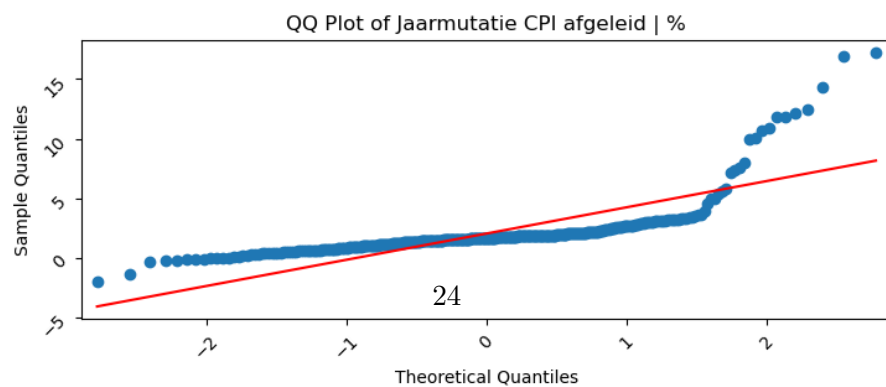
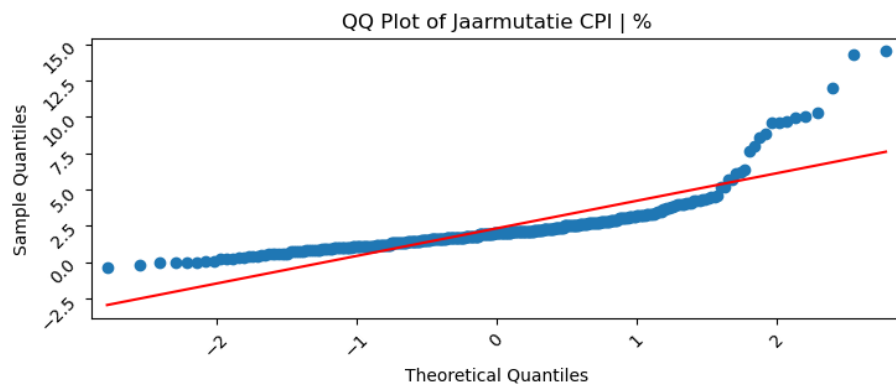
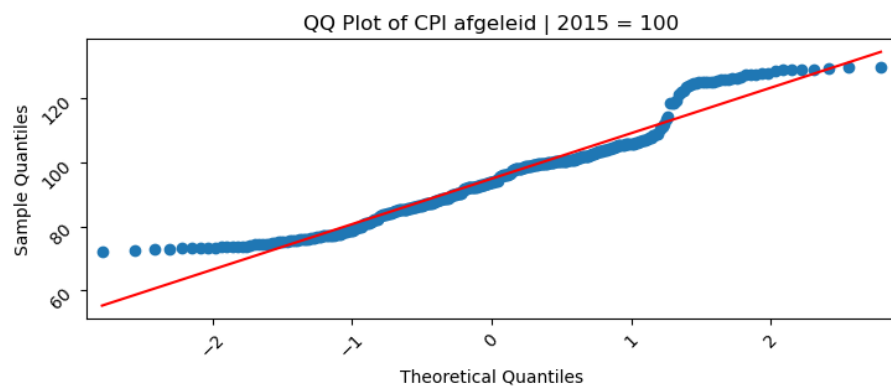
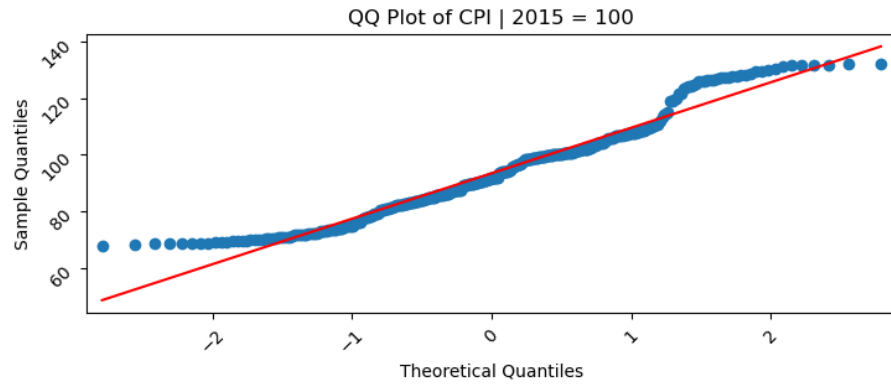
```
Amount of days with missing values: CPI | 2015 = 100          0.002639
CPI afgeleid | 2015 = 100          0.002639
Jaarmutatie CPI | %          0.036939
Jaarmutatie CPI afgeleid | %    0.036939
dtype: float64
```

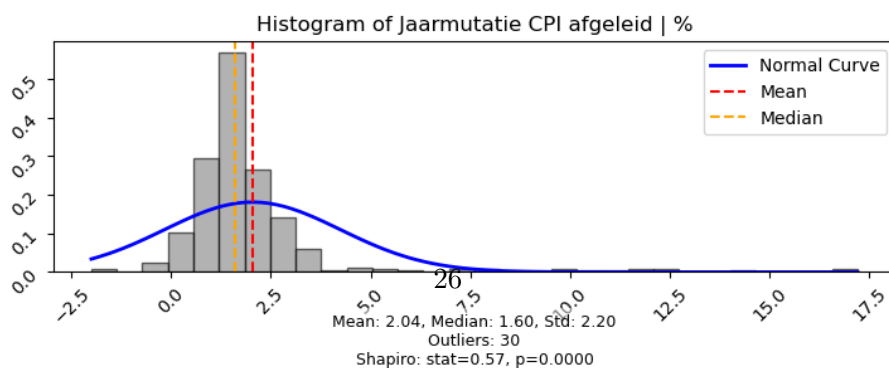
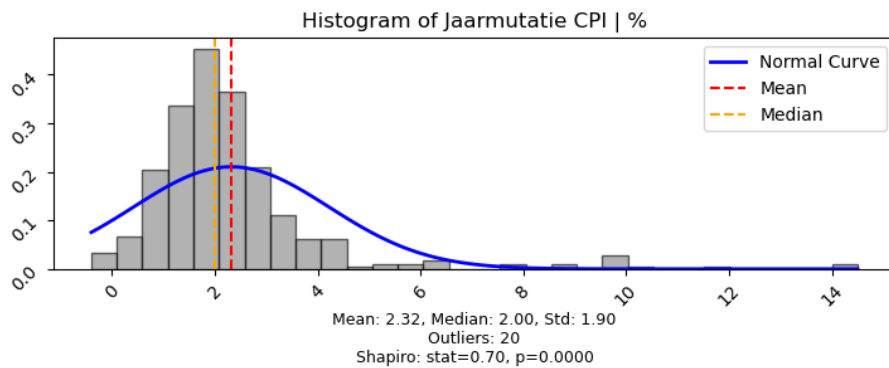
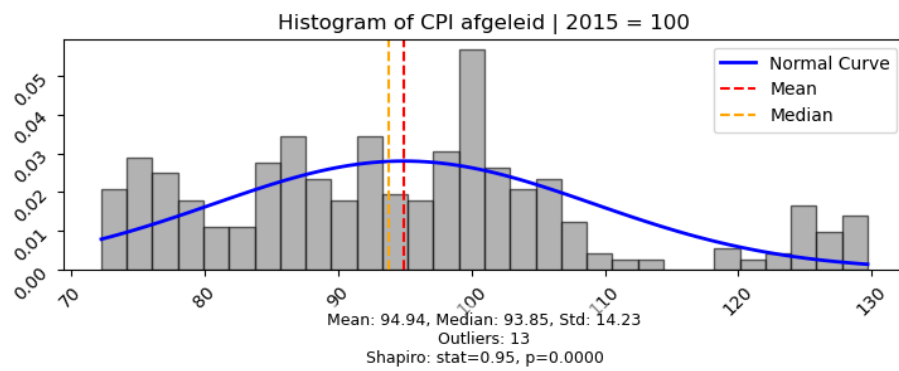
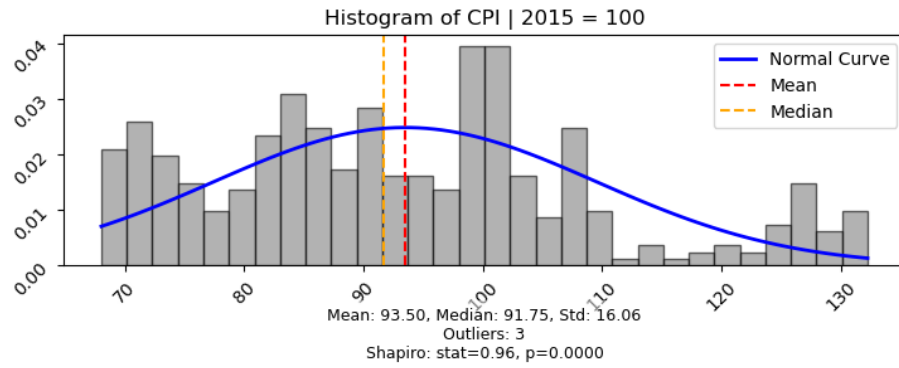
Descriptives of labor participation:

	CPI 2015 = 100	CPI afgeleid 2015 = 100	Jaarmutatie CPI % \
count	378.000000	378.000000	365.000000
mean	93.499471	94.936825	2.321918
std	16.062551	14.232265	1.899410
min	68.030000	72.280000	-0.400000
25%	82.265000	85.115000	1.300000
50%	91.745000	93.850000	2.000000
75%	102.022500	101.962500	2.700000
max	132.260000	129.680000	14.500000

```
Jaarmutatie CPI afgeleid | %  
count          365.000000  
mean           2.038904  
std            2.203648  
min            -2.000000  
25%            1.200000  
50%            1.600000  
75%            2.100000  
max            17.200000
```

Distribution and QQ plots:





6 Analysis 3 - GDP

```
[4]: Gdp = pd.read_csv('/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/
↳CBS/
↳Opbouw_binnenlands_product__bbp__nationale_rekeningen__1995_2022_28022025_221707.
↳csv', sep=';', header=2).T
Gdp.columns = Gdp.iloc[0]
Gdp = Gdp.reset_index(drop=True)
Gdp = Gdp.drop(columns=['Bron: CBS'])
Gdp = Gdp.drop([0,1])
Gdp = Gdp.rename(columns={'Onderwerp': 'Date'})
Gdp.set_index("Date", inplace=True)
Gdp.index = Gdp.index = pd.to_datetime(
    Gdp.index.astype(int), # Convert float to integer first
    format='%Y'           # Format specifier for 4-digit years
)
Gdp = Gdp.apply(pd.to_numeric, errors='coerce')

Create_EDA(Gdp)
```

Amount of days with missing values: Unnamed: 0

```
Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van
goederen en diensten (-)|Totaal      0.0
Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van
goederen en diensten (-)|Goederen    0.0
Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van
goederen en diensten (-)|Diensten    0.0
Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Bruto binnenlands
product                               0.0
dtype: float64
```

Descriptives of labor participation:

```
Unnamed: 0  Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer
van goederen en diensten (-)|Totaal  \
count                                28.000000
mean                                393949.964286
std                                  155646.527656
min                                  166743.000000
25%                                  269664.750000
50%                                  388207.500000
75%                                  497931.500000
max                                  795208.000000
```

Unnamed: 0 Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Goederen \

count	28.000000
mean	292291.678571
std	116885.242353
min	129241.000000
25%	201132.750000
50%	288000.500000
75%	353493.500000
max	621332.000000

Unnamed: 0 Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Diensten \

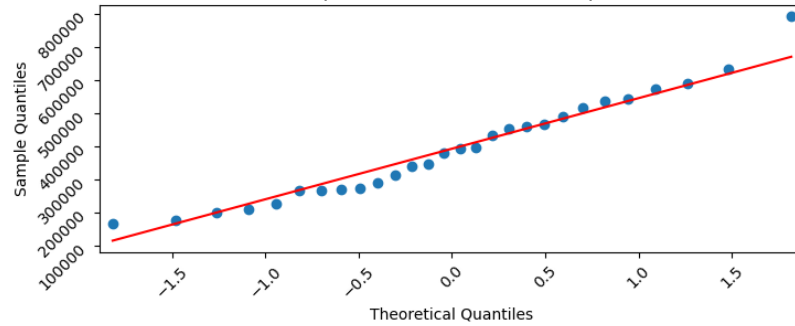
count	28.000000
mean	101658.285714
std	40695.443725
min	37502.000000
25%	72861.250000
50%	97514.000000
75%	138317.500000
max	173876.000000

Unnamed: 0 Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Bruto binnenlands product

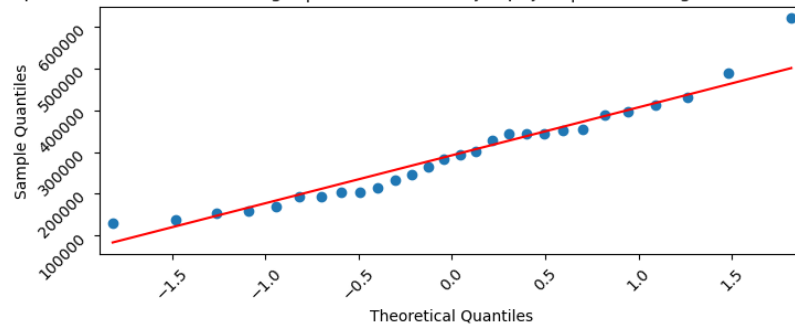
count	28.000000
mean	606588.071429
std	161030.055190
min	329547.000000
25%	496323.000000
50%	632014.500000
75%	694590.250000
max	958549.000000

Distribution and QQ plots:

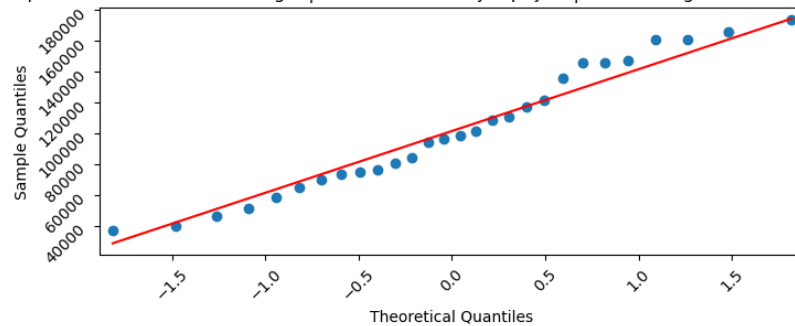
QQ Plot of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Totaal



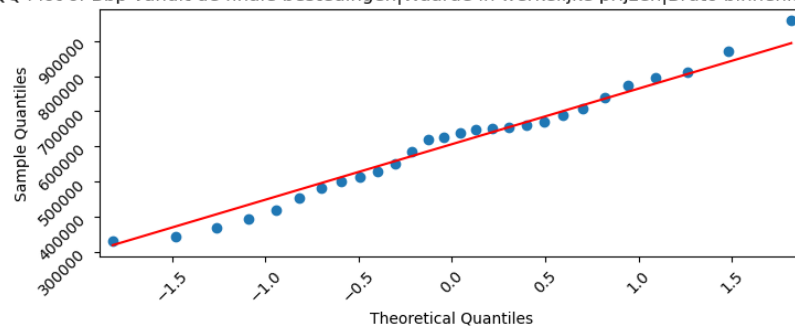
QQ Plot of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Goederen



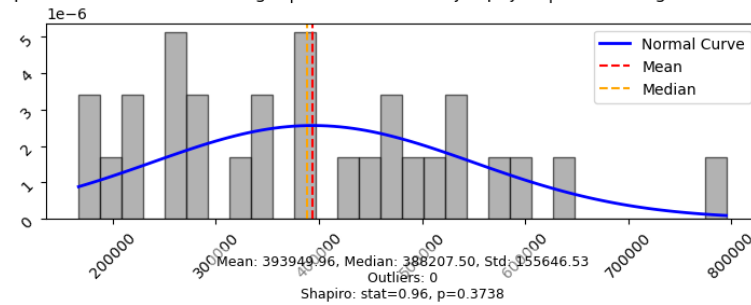
QQ Plot of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Diensten



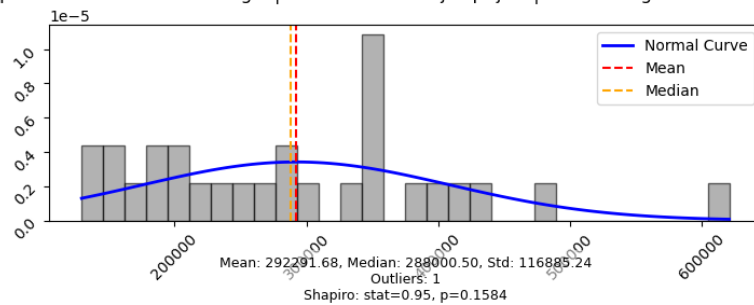
QQ Plot of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Bruto binnenlands product



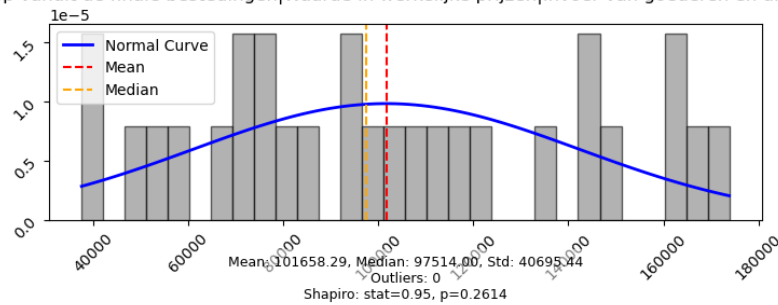
Histogram of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Totaal



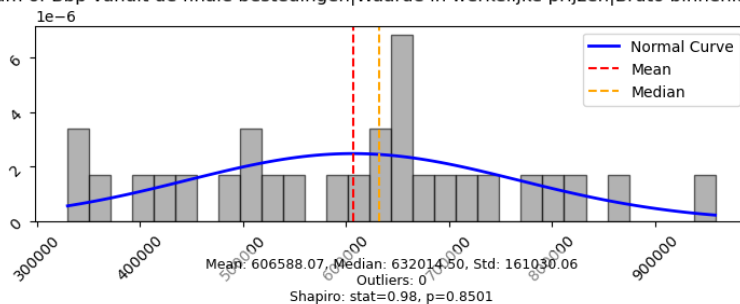
Histogram of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Goederen



Histogram of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Invoer van goederen en diensten (-)|Diensten



Histogram of Bbp vanuit de finale bestedingen|Waarde in werkelijke prijzen|Bruto binnenlands product



7 Analysis 4 - GDP Composition

```
[5]: GdpProductionAndSpending = pd.read_csv('/Users/ruben/Documents/GitHub/
↳MsCThesisRubenCuriel2024/Data/CBS/
↳Bbp__productie_en_bestedingen__kwartalen__mutaties__na__1995__2024_I_28022025_214426.
↳csv',sep=';', header=3).T
GdpProductionAndSpending = GdpProductionAndSpending.reset_index(drop=True)
# set header of
GdpProductionAndSpending.columns = GdpProductionAndSpending.iloc[0]

GdpProductionAndSpending = GdpProductionAndSpending.drop('Bron: CBS', axis=1)
GdpProductionAndSpending_daily =
↳expand_quarter_to_daily(GdpProductionAndSpending, 'Onderwerp')

GdpProductionAndSpending = convert_to_numeric(GdpProductionAndSpending)
GdpProductionAndSpending = GdpProductionAndSpending.apply(pd.to_numeric,
↳errors='coerce')

df_GdpProductionAndSpending = GdpProductionAndSpending_daily.
↳drop(['Onderwerp','date'],axis=1)
df_GdpProductionAndSpending = convert_to_numeric(df_GdpProductionAndSpending)
df_GdpProductionAndSpending.index.names = ['Date']

Create_EDA(df_GdpProductionAndSpending)
```

Amount of days with missing values: 0

Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Totaal
0.043224

Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Bruto binnenlands product
0.043224

Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale bestedingen|Bbp,
gecorrigeerd voor werkdageneffecten 0.538486

Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Invoer van goederen en diensten|Totaal
0.043224

Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Invoer van goederen en diensten|Goederen
0.043224

Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Invoer van goederen en diensten|Diensten
0.043224

Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Totaal

```

0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Totaal
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Consumptieve bestedingen|Totaal
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Consumptieve bestedingen|Huishoudens
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Consumptieve bestedingen|Overheid
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Bruto investeringen in vaste activa|Totaal
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Bruto investeringen in vaste activa|Bedrijven en huishoudens
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Bruto investeringen in vaste activa|Overheid
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Verandering in voorraden
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Uitvoer van goederen
en diensten|Totaal
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Uitvoer van goederen
en diensten|Goederen
0.043224
Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Uitvoer van goederen
en diensten|Diensten
0.043224
dtype: float64

```

Descriptives of labor participation:

```

0      Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Totaal  \
count      18173.000000
mean      1.752352
std      3.123635
min     -10.000000
25%      0.400000
50%      1.400000
75%      3.500000
max     15.000000

```

```

0      Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale
bestedingen|Bruto binnenlands product  \

```


count	18173.000000
mean	1.123854
std	2.020724
min	-8.800000
25%	0.200000
50%	0.900000
75%	2.200000
max	6.400000

0 Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale bestedingen|Bbp, gecorrigeerd voor werkdageneffecten \

count	8766.000000
mean	1.724584
std	2.379672
min	-8.500000
25%	0.500000
50%	2.000000
75%	3.375000
max	5.800000

0 Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale bestedingen|Invoer van goederen en diensten|Totaal \

count	18173.000000
mean	2.844192
std	5.549926
min	-18.100000
25%	0.600000
50%	2.300000
75%	5.300000
max	34.000000

0 Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale bestedingen|Invoer van goederen en diensten|Goederen \

count	18173.000000
mean	2.906724
std	4.397411
min	-14.700000
25%	0.800000
50%	2.200000
75%	5.400000
max	16.200000

0 Opbouw bbp vanuit de finale bestedingen|Beschikbaar voor finale bestedingen|Invoer van goederen en diensten|Diensten \

count	18173.000000
mean	3.212750
std	13.464676
min	-46.500000

25%	-0.100000
50%	2.000000
75%	5.500000
max	111.100000

0	Opbouw bbp vanuit de finale bestedingen Finale bestedingen Totaal \
count	18173.000000
mean	1.752352
std	3.123635
min	-10.000000
25%	0.400000
50%	1.400000
75%	3.500000
max	15.000000

0	Opbouw bbp vanuit de finale bestedingen Finale bestedingen Nationale finale bestedingen Totaal \
count	18173.000000
mean	1.084549
std	3.785360
min	-15.800000
25%	-0.200000
50%	0.700000
75%	2.400000
max	27.000000

0	Opbouw bbp vanuit de finale bestedingen Finale bestedingen Nationale finale bestedingen Consumptieve bestedingen Totaal \
count	18173.000000
mean	0.937545
std	1.713593
min	-8.100000
25%	0.100000
50%	0.600000
75%	1.600000
max	6.700000

0	Opbouw bbp vanuit de finale bestedingen Finale bestedingen Nationale finale bestedingen Consumptieve bestedingen Huishoudens \
count	18173.000000
mean	0.751698
std	2.274406
min	-12.600000
25%	-0.100000
50%	0.600000
75%	1.400000
max	8.400000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale bestedingen|Consumptieve bestedingen|Overheid \

count	18173.000000
mean	1.309784
std	1.876365
min	-2.100000
25%	0.200000
50%	0.700000
75%	2.300000
max	9.100000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale bestedingen|Bruto investeringen in vaste activa|Totaal \

count	18173.000000
mean	2.295438
std	16.080610
min	-47.100000
25%	-1.500000
50%	1.200000
75%	4.400000
max	161.100000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale bestedingen|Bruto investeringen in vaste activa|Bedrijven en huishoudens \

count	18173.000000
mean	2.772046
std	20.771558
min	-52.300000
25%	-2.100000
50%	1.500000
75%	4.700000
max	219.200000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale bestedingen|Bruto investeringen in vaste activa|Overheid \

count	18173.000000
mean	1.353937
std	4.805712
min	-13.700000
25%	-1.800000
50%	0.800000
75%	3.700000
max	16.200000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale bestedingen|Verandering in voorraden \

count	18173.000000
-------	--------------

mean	0.155406
std	0.647201
min	-1.700000
25%	-0.300000
50%	0.200000
75%	0.600000
max	2.100000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Uitvoer van goederen en diensten|Totaal \

count	18173.000000
mean	2.714323
std	4.166972
min	-12.300000
25%	0.900000
50%	2.200000
75%	5.200000
max	13.000000

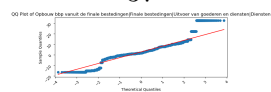
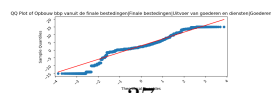
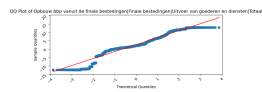
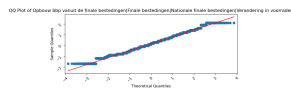
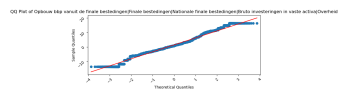
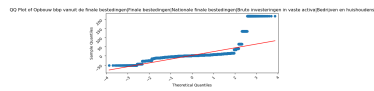
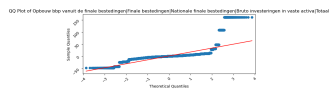
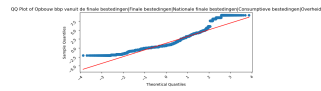
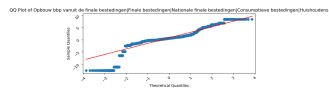
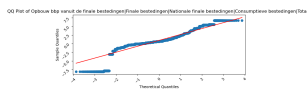
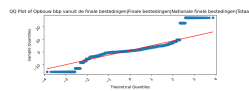
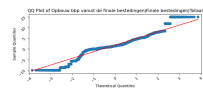
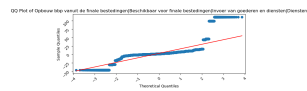
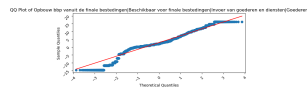
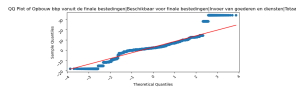
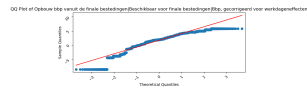
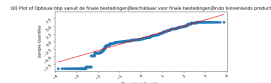
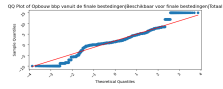
0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Uitvoer van goederen en diensten|Goederen \

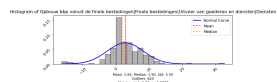
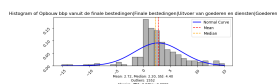
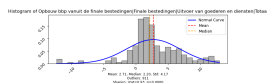
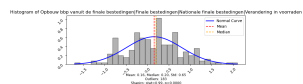
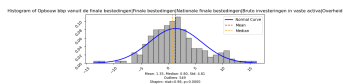
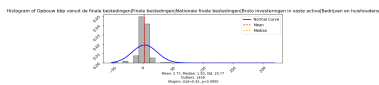
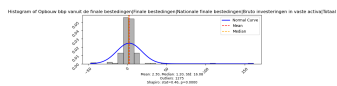
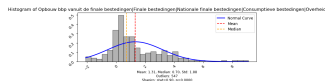
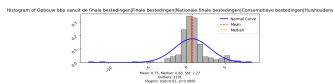
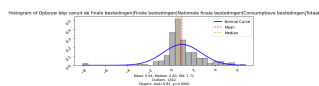
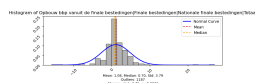
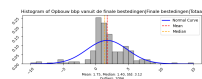
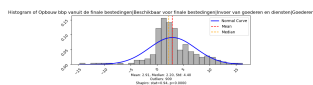
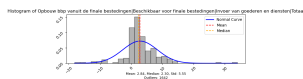
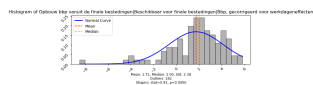
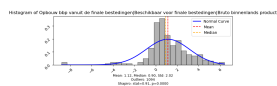
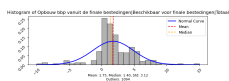
count	18173.000000
mean	2.723689
std	4.396232
min	-15.200000
25%	0.700000
50%	2.200000
75%	4.800000
max	15.000000

0 Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Uitvoer van goederen en diensten|Diensten

count	18173.000000
mean	2.836906
std	5.499112
min	-16.400000
25%	0.300000
50%	1.900000
75%	5.800000
max	32.800000

Distribution and QQ plots:





8 Analysis 5 - Housing

```
[6]: Boughthomes = pd.read_csv('/Users/ruben/Documents/GitHub/
↳MsCThesisRubenCuriel2024/Data/CBS/
↳Bestaande_koopwoningen_verkooprijzen_prijsindex_2015_100_1995_2023_25022025_192415.
↳csv',sep=';', header=3)

# Boughthomes_daily = expand_year_month_to_daily(Boughthomes,
↳year_month_col="Unnamed: 0")
Boughthomes['Unnamed: 0'] = Boughthomes['Unnamed: 0'].apply(lambda x:
↳convert_year_month_string(x))
Boughthomes.rename(columns={'Unnamed: 0':'Date'}, inplace=True)
Boughthomes.set_index(['Date'], inplace=True)
Boughthomes = Boughthomes.iloc[1:]
Boughthomes = convert_to_numeric(Boughthomes)
df_Boughthomes = Boughthomes.apply(pd.to_numeric, errors='coerce')

Create_EDA(df_Boughthomes)
```

Amount of days with missing values: Prijsindex verkooprijzen|Prijsindex
bestaande koopwoningen 0.002646

Prijsindex verkooprijzen Ontwikkeling t.o.v. voorgaande periode	0.007937
Prijsindex verkooprijzen Ontwikkeling t.o.v. een jaar eerder	0.037037
Aantal verkochte woningen Aantal verkochte woningen	0.002646
Aantal verkochte woningen Ontwikkeling t.o.v. voorgaande periode	0.007937
Aantal verkochte woningen Ontwikkeling t.o.v. een jaar eerder	0.037037
Gemiddelde verkoopprijs	0.002646
Totale waarde verkooprijzen	0.002646

dtype: float64

Descriptives of labor participation:

	Prijsindex verkooprijzen Prijsindex bestaande koopwoningen \
count	377.000000
mean	105.791777
std	34.063191
min	41.300000
25%	94.000000
50%	105.300000
75%	117.800000
max	189.200000

	Prijsindex verkooprijzen Ontwikkeling t.o.v. voorgaande periode \
count	375.000000

mean	0.812800
std	2.357261
min	-6.600000
25%	0.000000
50%	0.500000
75%	1.000000
max	18.200000

	Prijsindex verkoopprijzen Ontwikkeling t.o.v. een jaar eerder \
count	364.000000
mean	5.520055
std	6.795932
min	-9.900000
25%	2.175000
50%	4.650000
75%	10.125000
max	21.100000

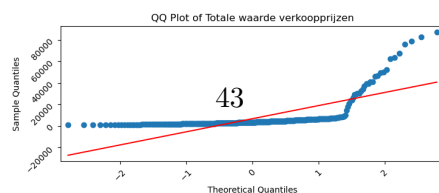
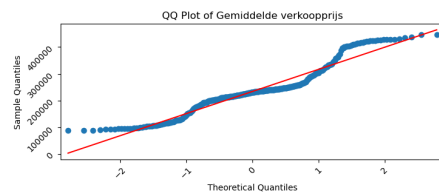
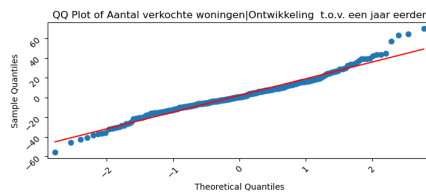
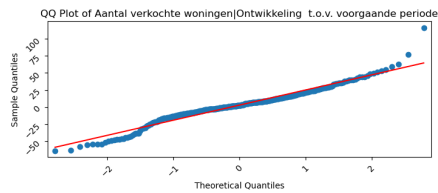
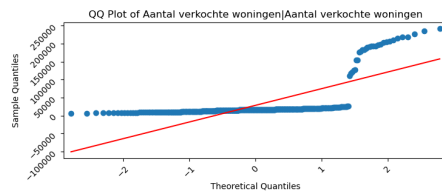
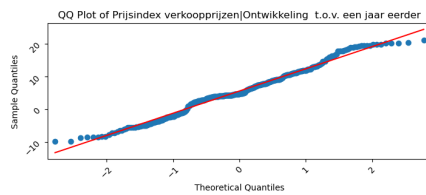
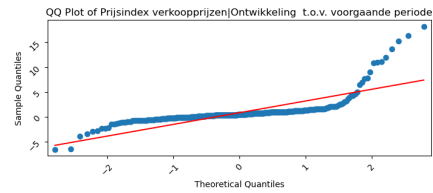
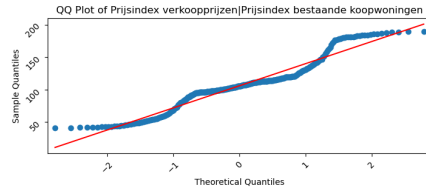
	Aantal verkochte woningen Aantal verkochte woningen \
count	377.000000
mean	28368.859416
std	46306.329299
min	6260.000000
25%	13266.000000
50%	16048.000000
75%	18474.000000
max	241860.000000

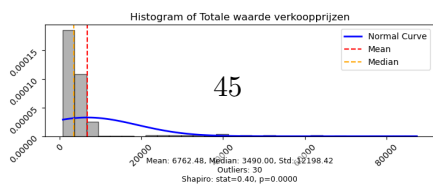
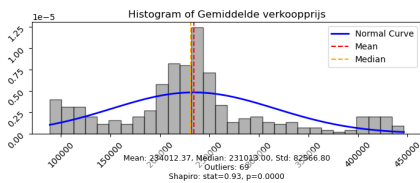
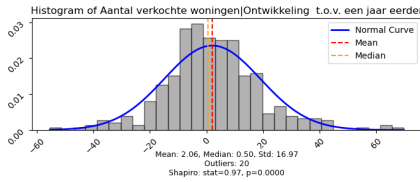
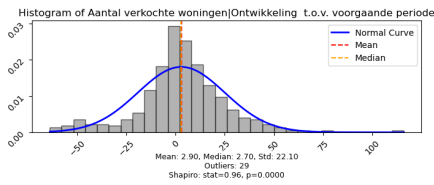
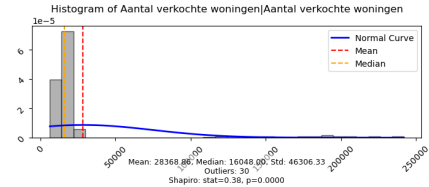
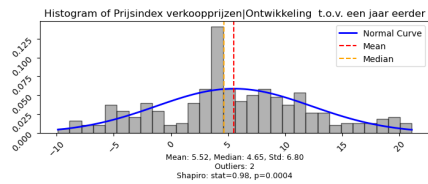
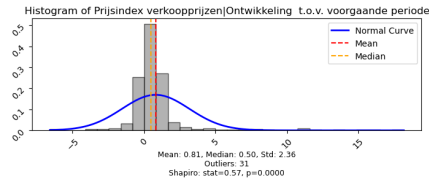
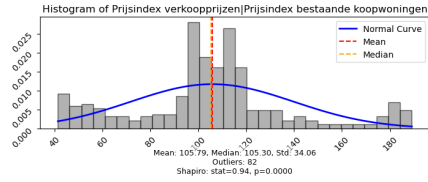
	Aantal verkochte woningen Ontwikkeling t.o.v. voorgaande periode \
count	375.000000
mean	2.897067
std	22.098140
min	-63.900000
25%	-7.400000
50%	2.700000
75%	14.050000
max	116.000000

	Aantal verkochte woningen Ontwikkeling t.o.v. een jaar eerder \
count	364.000000
mean	2.055495
std	16.974057
min	-55.400000
25%	-7.700000
50%	0.500000
75%	10.600000
max	69.800000

	Gemiddelde verkoopprijs	Totale waarde verkoopprijzen
count	377.000000	377.000000
mean	234012.366048	6762.482759
std	82566.799589	12198.415837
min	88991.000000	782.000000
25%	197440.000000	2345.000000
50%	231013.000000	3490.000000
75%	256800.000000	5442.000000
max	446263.000000	87431.000000

Distribution and QQ plots:





9 Analysis 6 - Commodities

```
[7]: def find_columns_energy_commodities(df):
    # Define a pattern that matches any of the keywords
    pattern = r"(Coal|Coal (Colombia)|Coal (South Africa)|Crude oil| index_
↳(Laspeyres)|Natural Gas|natural gas)"
    # Use a list comprehension to check each column name
    matching_columns = [col for col in df.columns if re.search(pattern,
↳str(col), re.IGNORECASE)]

    return matching_columns

Commodities = pd.read_excel('/Users/ruben/Documents/GitHub/
↳MsCThesisRubenCuriel2024/Data/Commodity Future Data/CMO/
↳CMO-Historical-Data-Monthly.xlsx', sheet_name='Monthly Prices', skiprows=4)
Commodities['Unnamed: 0'] = Commodities['Unnamed: 0'].apply(lambda x:
↳convert_year_month_string(x))
Commodities.set_index('Unnamed: 0', inplace=True)
Commodities.index.names = ['Date']
Commodities.columns += ' | ' + Commodities.iloc[0]
Commodities = Commodities.iloc[1:]

df_Commodities = Commodities[find_columns_energy_commodities(Commodities)]
df_Commodities = df_Commodities.apply(pd.to_numeric, errors='coerce')
Create_EDA(df_Commodities)
```

Amount of days with missing values: Crude oil, average | (\$/bbl)

0.000000

Crude oil, Brent (\$/bbl)	0.000000
Crude oil, Dubai (\$/bbl)	0.000000
Crude oil, WTI (\$/bbl)	0.338462
Coal, Australian (\$/mt)	0.153846
Coal, South African ** (\$/mt)	0.369231
Natural gas, US (\$/mmbtu)	0.000000
Natural gas, Europe (\$/mmbtu)	0.000000
Liquefied natural gas, Japan (\$/mmbtu)	0.261538
Natural gas index (2010=100)	0.261538

dtype: float64

Descriptives of labor participation:

	Crude oil, average (\$/bbl)	Crude oil, Brent (\$/bbl) \
count	780.000000	780.000000
mean	33.412984	34.230560

std	31.357458	32.522904
min	1.210000	1.210000
25%	11.510000	11.510000
50%	22.140833	22.565000
75%	52.857500	54.127273
max	132.825184	133.873043

	Crude oil, Dubai (\$/bbl)	Crude oil, WTI (\$/bbl) \
count	780.000000	516.000000
mean	32.763726	45.980657
std	31.653220	28.602558
min	1.210000	11.310000
25%	10.600000	20.655000
50%	20.835000	33.072950
75%	51.482625	69.567841
max	131.224783	133.927143

	Coal, Australian (\$/mt)	Coal, South African ** (\$/mt) \
count	660.000000	492.000000
mean	57.747535	60.478932
std	55.007139	43.592495
min	7.800000	21.250000
25%	29.250000	30.384375
50%	38.375000	43.956250
75%	68.967500	82.915625
max	430.810000	302.000000

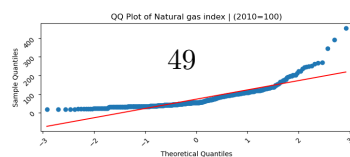
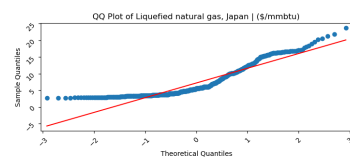
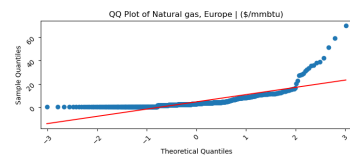
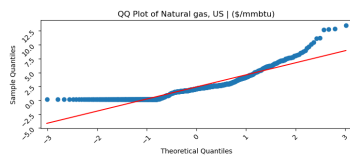
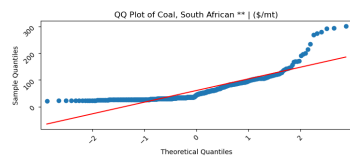
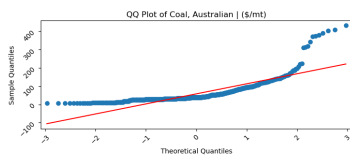
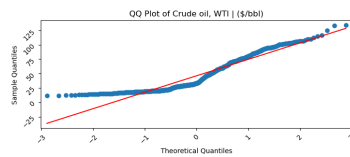
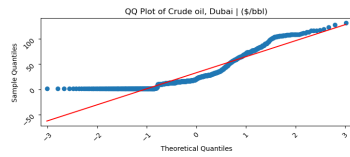
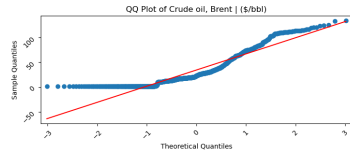
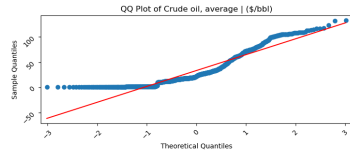
	Natural gas, US (\$/mmbtu)	Natural gas, Europe (\$/mmbtu) \
count	780.000000	780.000000
mean	2.399592	4.782296
std	2.175583	6.192197
min	0.140000	0.384150
25%	0.547500	1.735113
50%	2.100000	3.020000
75%	3.024675	5.857080
max	13.522619	70.043640

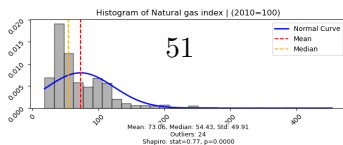
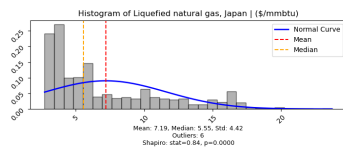
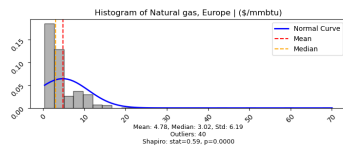
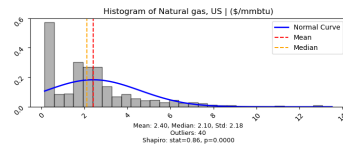
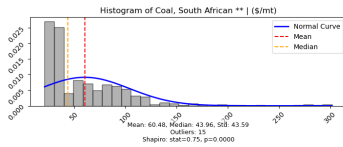
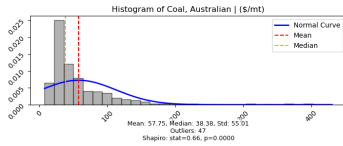
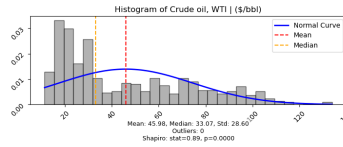
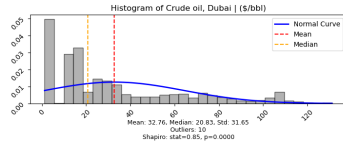
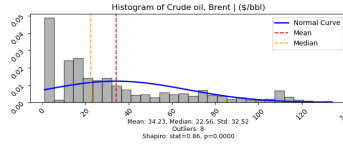
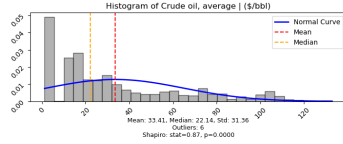
	Liquefied natural gas, Japan (\$/mmbtu) \
count	576.000000
mean	7.189415
std	4.421399
min	2.720000
25%	3.640000
50%	5.545816
75%	10.012411
max	23.733791

Natural gas index | (2010=100)

count	576.000000
mean	73.060097
std	49.906595
min	18.530831
25%	40.031983
50%	54.430696
75%	95.329345
max	454.040143

Distribution and QQ plots:





10 Analysis 7 - SST

```
[8]: import netCDF4 as nc
import pandas as pd
import datetime

# read
path = '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SST/sst.nc'
ds = nc.Dataset(path)

# get variables
time = ds['TIME'][:].data
nat = ds['NAT'][:].data

# create df
nat_df = pd.DataFrame({'time':time, 'nat':nat})

# create date column based on fact that time is the distance in days from
↳1800-01-01
start_date = datetime.date(1800, 1, 1)
nat_df['Date'] = nat_df['time'].apply(lambda x: start_date + datetime.
↳timedelta(days=x))
nat_df['Date'] = pd.to_datetime(nat_df['Date'])

nat_df.drop(columns=['time'], inplace=True)

# sst df is missing dates, create range of all dates and merge this to the df
min_date, max_date = nat_df['Date'].min(), nat_df['Date'].max()
date_range = pd.date_range(start=min_date, end=max_date, freq='D')
df_dates = pd.DataFrame(date_range, columns=['Date'])
df_dates['Date'] = pd.to_datetime(df_dates['Date'])

nat_df_all_dates = df_dates.merge(nat_df, on = 'Date', how = 'left')

# back fill the empty days
nat_df_filled = nat_df_all_dates.fillna(method='bfill')

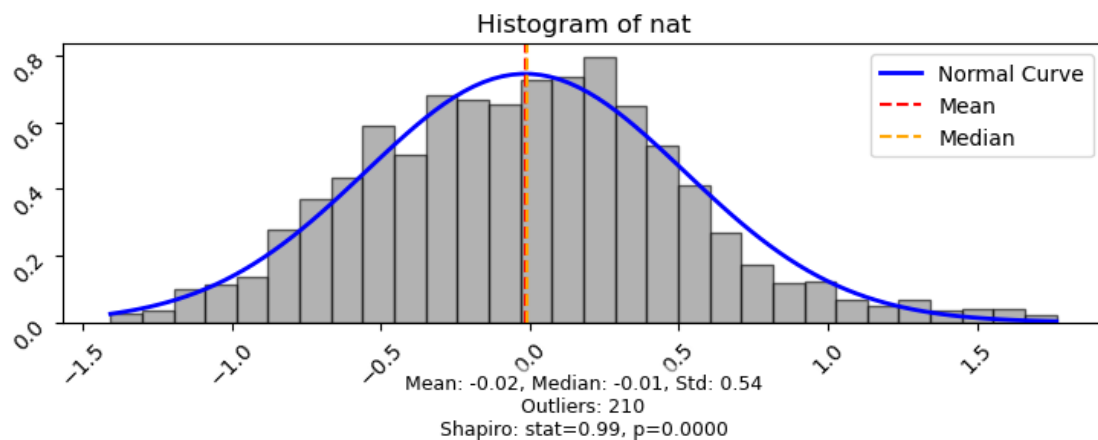
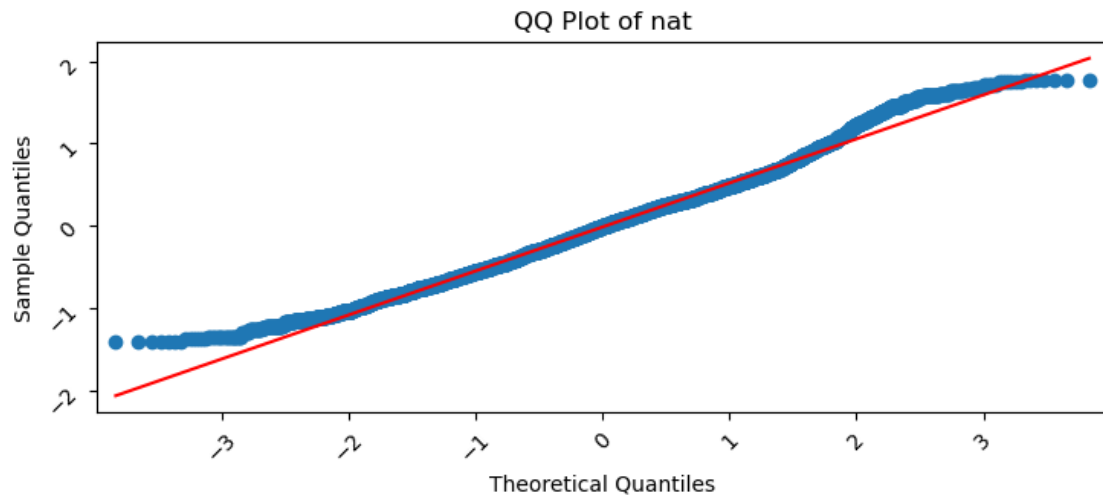
nat_df_filled = nat_df_filled.set_index('Date')
nat_df_filled = nat_df_filled.apply(pd.to_numeric, errors='coerce')

nat_df_filled.index.names = ['Date']
```

```

nat_QQ_Plots = Create_QQ_Plot(nat_df_filled)
nat_Distribution_Plots = _
↳ Create_Hist_Normal_Plot_With_Stats_Outliers(nat_df_filled)

```



11 Analysis 8 - NAO

```

[9]: nao_path = '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/NAO.txt'
nao = preprocess_nao(nao_path)

Create_EDA(nao)

```

```

Amount of days with missing values: Year      0.000000
Month      0.000000
NAO        0.012061

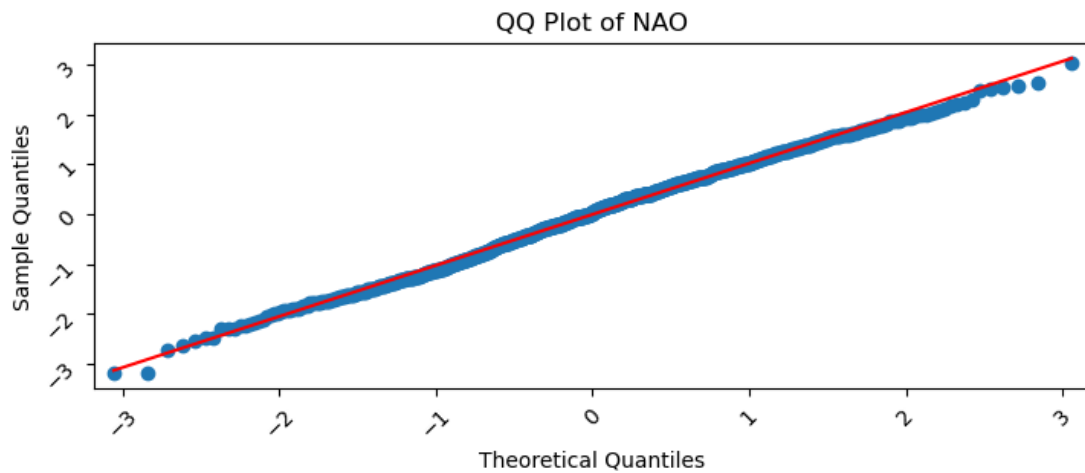
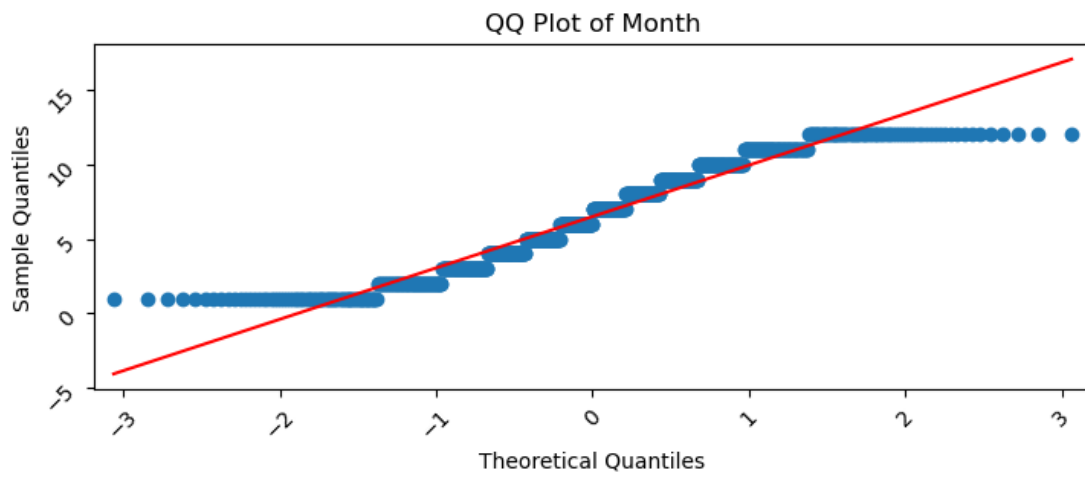
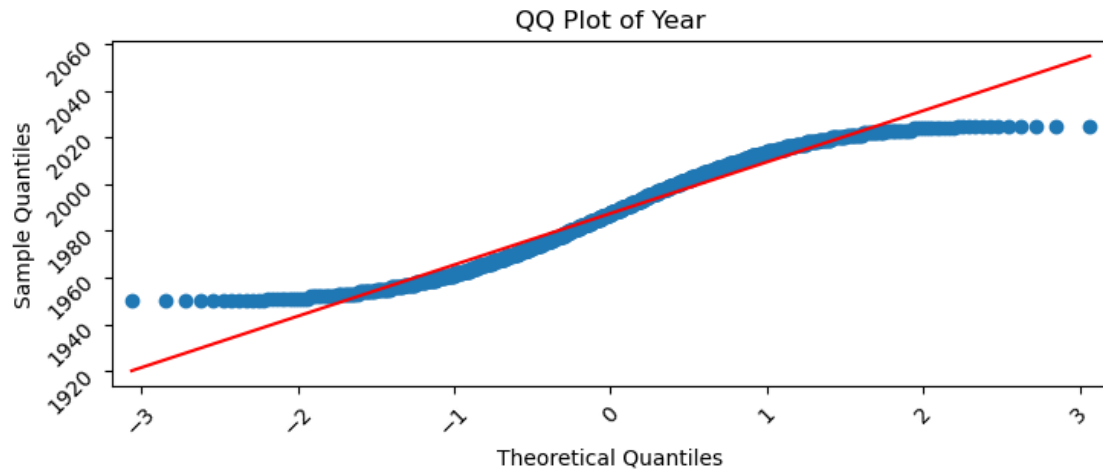
```

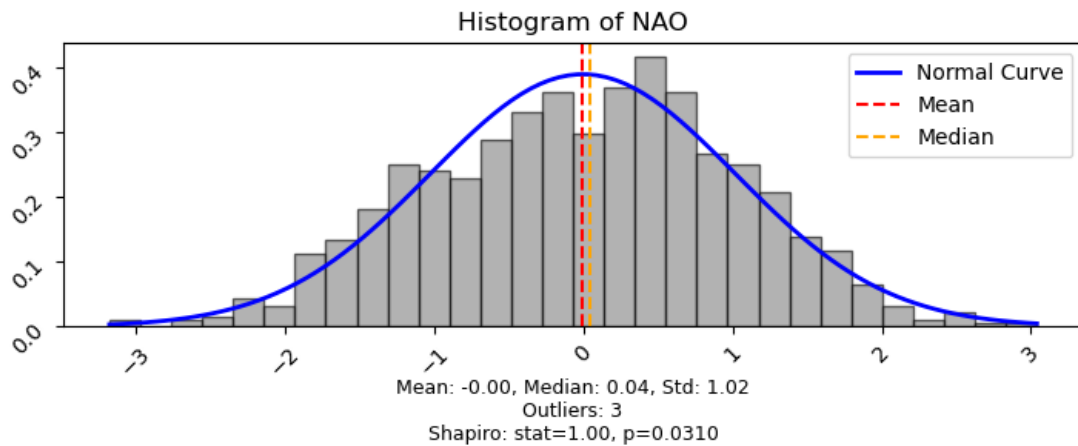
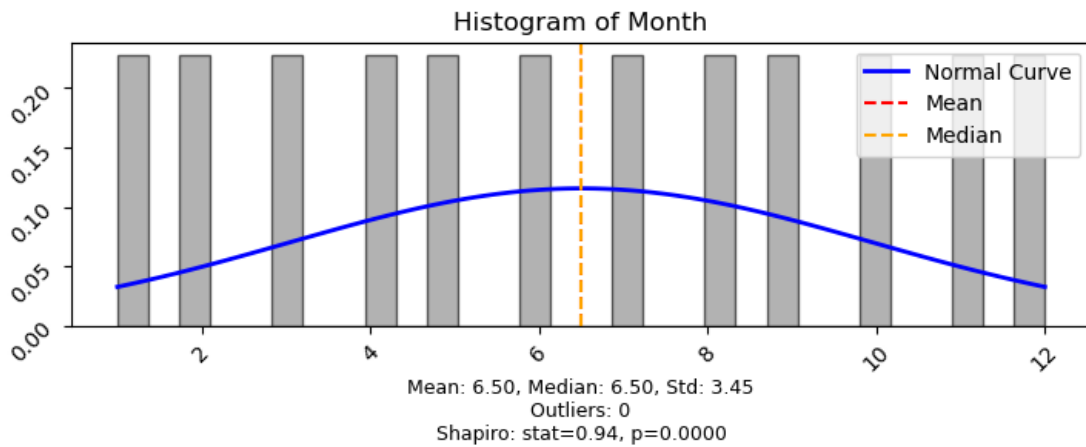
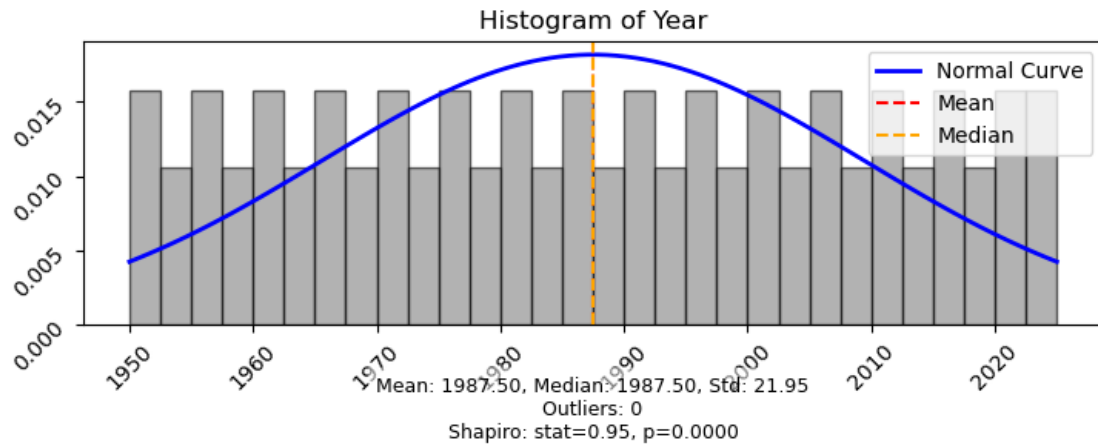
dtype: float64

Descriptives of labor participation:

	Year	Month	NAO
count	912.000000	912.000000	901.000000
mean	1987.500000	6.500000	-0.004950
std	21.949448	3.453947	1.021459
min	1950.000000	1.000000	-3.180000
25%	1968.750000	3.750000	-0.750000
50%	1987.500000	6.500000	0.040000
75%	2006.250000	9.250000	0.720000
max	2025.000000	12.000000	3.040000

Distribution and QQ plots:





12 Analysis 8 - Weather

```
[10]: file_path = '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/KNMI/
      ↪etmgeg_260.txt'
      weather = preprocess_weather(file_path)

      Create_EDA(weather)
```

```
Amount of days with missing values: Date          0.0
DailyPrecipitation          0.0
MaxHourlyPrecipitation      0.0
HDMaxPrecipitation          0.0
DailyMeanTemperature        0.0
HourlyMinTemperature        0.0
HDMINTemperature           0.0
HourlyMaxTemperature        0.0
HDMaxTemperature           0.0
DailyMean Windspeed         0.0
MaxHourlyMean Windspeed     0.0
HDMaxMean Windspeed         0.0
MinHourlyMean Windspeed     0.0
HDMINMean Windspeed         0.0
Month                       0.0
Year                       0.0
Week                       0.0
WeekKey                    0.0
day_of_week                0.0
dtype: float64
```

Descriptives of labor participation:

	Date	HDMaxPrecipitation	\
count	1512	1512.000000	
mean	2023-01-26 11:59:59.999999744	7.007275	
min	2021-01-01 00:00:00	1.000000	
25%	2022-01-13 18:00:00	1.000000	
50%	2023-01-26 12:00:00	1.000000	
75%	2024-02-08 06:00:00	13.000000	
max	2025-02-20 00:00:00	24.000000	
std	NaN	7.783849	

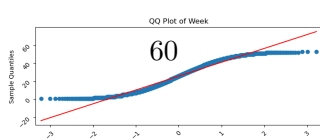
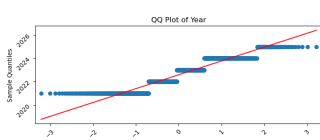
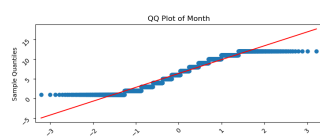
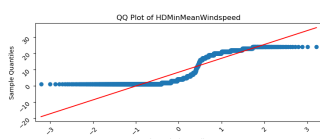
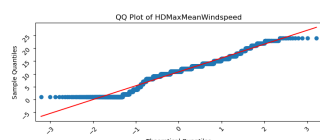
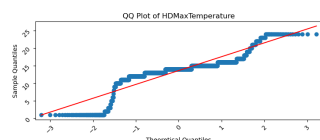
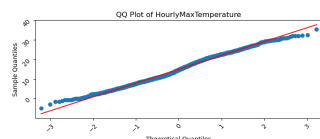
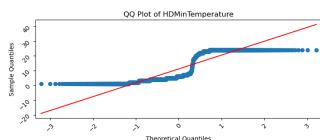
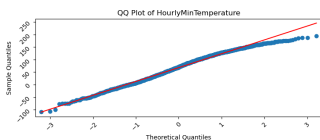
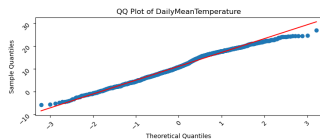
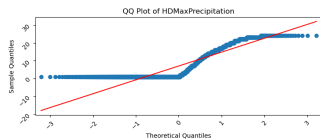
	DailyMeanTemperature	HourlyMinTemperature	HDMINTemperature	\
count	1512.000000	1512.000000	1512.000000	
mean	11.122222	67.980820	11.107804	

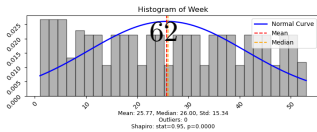
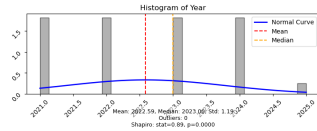
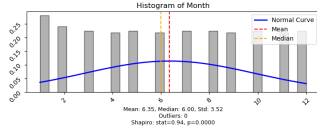
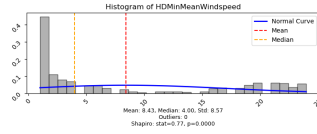
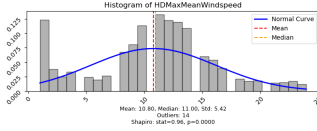
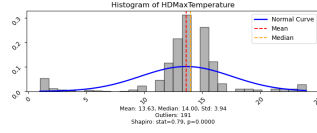
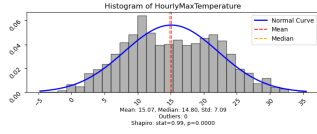
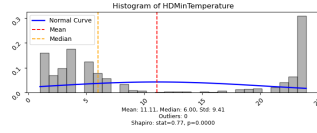
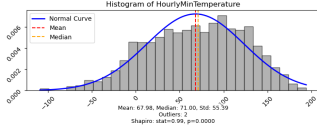
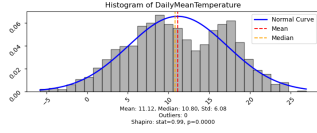
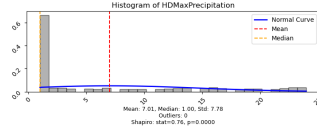
min	-5.800000	-109.000000	1.000000
25%	6.700000	26.000000	3.750000
50%	10.800000	71.000000	6.000000
75%	16.300000	112.000000	23.000000
max	26.900000	194.000000	24.000000
std	6.080511	55.388457	9.412716

	HourlyMaxTemperature	HDMaxTemperature	HDMaxMean Windspeed \
count	1512.000000	1512.000000	1512.000000
mean	15.074008	13.628307	10.796296
min	-4.800000	1.000000	1.000000
25%	9.700000	13.000000	8.000000
50%	14.800000	14.000000	11.000000
75%	20.800000	15.000000	14.000000
max	35.500000	24.000000	24.000000
std	7.089871	3.941423	5.418383

	HDMINMeanWindspeed	Month	Year	Week
count	1512.000000	1512.000000	1512.000000	1512.0
mean	8.426587	6.349868	2022.585317	25.774471
min	1.000000	1.000000	2021.000000	1.0
25%	1.000000	3.000000	2022.000000	12.0
50%	4.000000	6.000000	2023.000000	26.0
75%	18.000000	9.000000	2024.000000	39.0
max	24.000000	12.000000	2025.000000	53.0
std	8.566084	3.516617	1.188683	15.341817

Distribution and QQ plots:





13 Analysis 9 - Energy Prices

```
[11]: def preprocess_energy_price(energy_prices_path: str) -> pd.DataFrame:
    energy_prices = pd.read_csv(energy_prices_path, delimiter=';')
    # Apply the function to create monthly entries
    energy_prices['Date'] = energy_prices['Perioden'].
    ↪ apply(expand_halfjaar_to_months)

    # Explode the list of months into individual rows
    energy_prices = energy_prices.explode('Date')

    # Convert to datetime and set as index
    energy_prices['Date'] = pd.to_datetime(energy_prices['Date'])
    energy_prices.set_index('Date', inplace=True)

    # Optional: Drop the original 'period' column if no longer needed
    energy_prices = energy_prices.drop(columns=['Perioden', 'Prijscomponenten'])

    # Convert to numeric
    energy_prices = convert_to_numeric(energy_prices)

    return energy_prices

energy_prices_path = '/Users/ruben/Downloads/
    ↪ Eindverbruikersprijzen_aardgas_en_elektriciteit_23032025_103045.csv'

energy_prices = preprocess_energy_price(energy_prices_path)
Create_EDA(energy_prices)
```

```
Amount of days with missing values: euro per m3      0.0
euro per m3.1      0.0
euro per m3.2      0.0
euro per kWh      0.0
euro per kWh.1     0.0
euro per kWh.2     0.0
dtype: float64
```

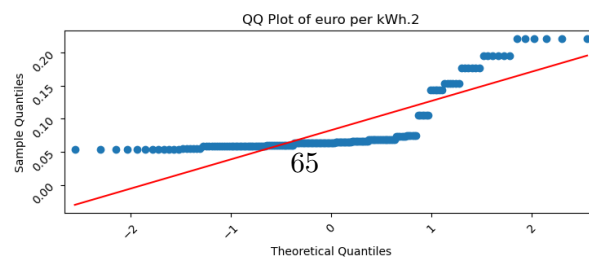
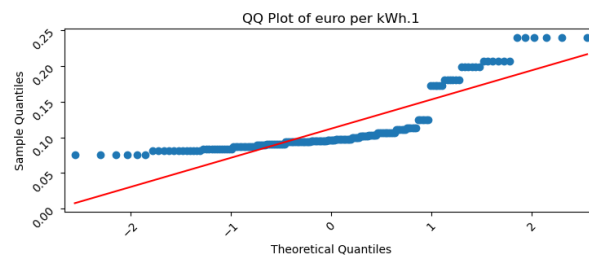
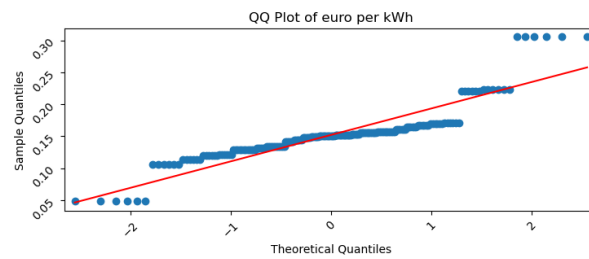
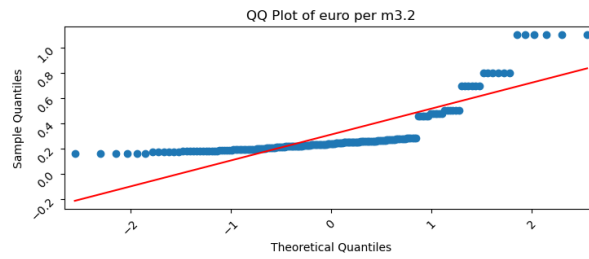
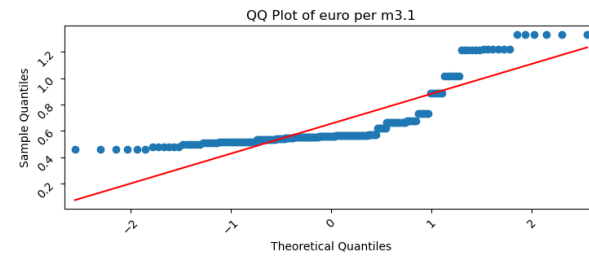
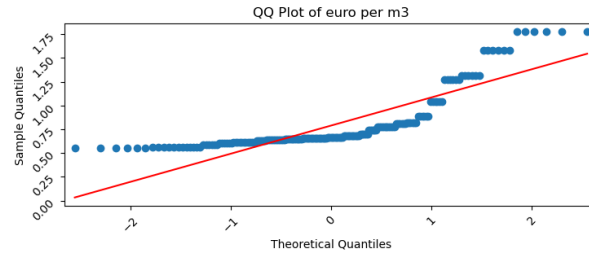
Descriptives of labor participation:

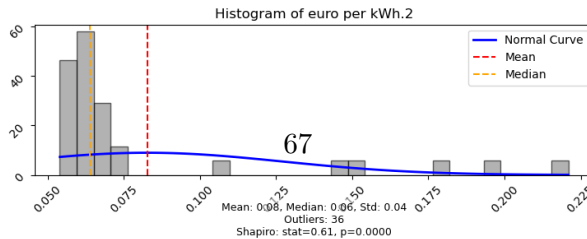
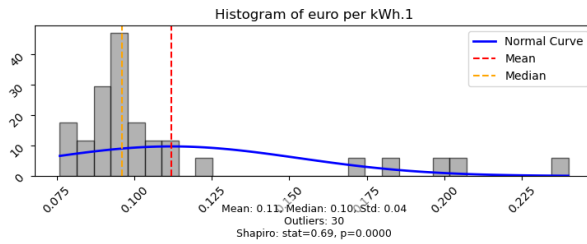
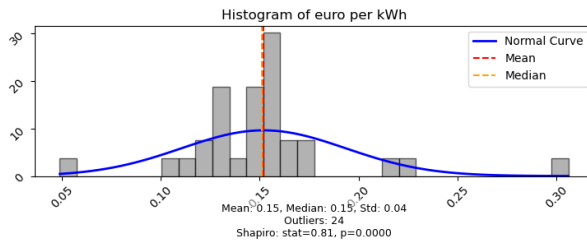
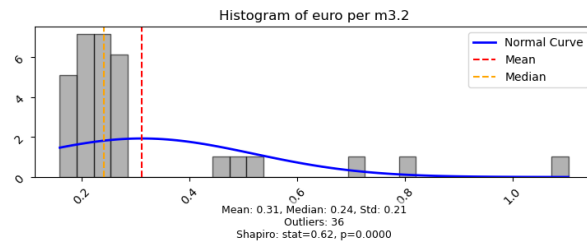
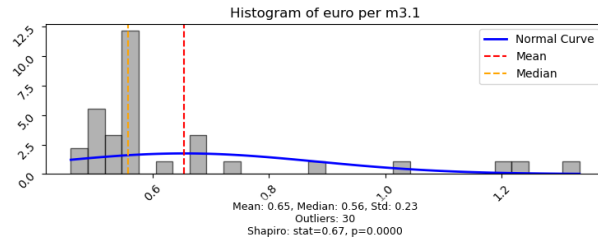
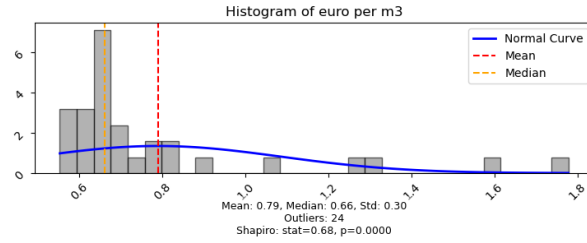
	euro per m3	euro per m3.1	euro per m3.2	euro per kWh \
count	186.000000	186.000000	186.000000	186.000000
mean	0.789032	0.654097	0.311452	0.151968
std	0.296575	0.227656	0.206715	0.041523
min	0.553000	0.459000	0.159000	0.049000

25%	0.630000	0.533000	0.202000	0.131000
50%	0.662000	0.558000	0.241000	0.151000
75%	0.809000	0.666000	0.279000	0.160000
max	1.778000	1.334000	1.104000	0.306000

	euro per kWh.1	euro per kWh.2
count	186.000000	186.000000
mean	0.112000	0.082677
std	0.040915	0.044402
min	0.076000	0.054000
25%	0.089000	0.059000
50%	0.096000	0.064000
75%	0.111000	0.073000
max	0.240000	0.221000

Distribution and QQ plots:





14 Analysis 10 - SIC

```
[12]: # get paths of sea ice concentration files
sic_paths = [
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪1ede5264bd6ecbc3cefa0e2eb5651a88/',
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪8e7fc0e0427d9bbbbb4f932132100311',
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪9f0afe1ca7db97892fb656304822dd29/',
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪717a61014ce7c5e1035ff279ac05004e/',
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪b11b2038dfe57c16035b04ef8448ef88/',
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪d1a23a892042fa09d03d711914b8f32d/',
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪e5ac6ede289daf14a04b4d8f6aad5c27/' ,
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/SIC/
    ↪e66b0b4a5685e0113270a0e05eaa3267/'
]
# create empty dictionary
sic_dict = {}

for path in sic_paths:
    sea_ice_concentration_files = os.listdir(path)

    # iterate through the files, use try expect because not all files work
    for sic_file in sea_ice_concentration_files:
        try:
            # open file as nc dataset
            ds = nc.Dataset(path + sic_file)

            # get time and ice concentration
            time = ds.variables['time'][:]
            ice_conc = ds.variables['ice_conc'][:]

            # create df from ice concentration and select non nans
            df = pd.DataFrame({'sic': ice_conc.flatten()})
            sic = np.array(df[df['sic'].notna()]['sic']).reshape(-1, 1)

            # apply pca to reduce size, we had a lot of values due to multiple
            ↪longitudes x latitudes
```

```

pca = PCA(n_components=1)
pca.fit(sic)

# add to dict
sic_dict[time[0]] = pca.singular_values_[0]
except:
    continue

# create df from dict
sic_df = pd.DataFrame(list(sic_dict.items()), columns=['time', 'sic'])

# get date from time (time is distance in seconds from 1978-01-01)
sic_df['date'] = sic_df['time'].apply(lambda x: datetime.date(1978, 1, 1) +
    ↪datetime.timedelta(0, x))
sic_df['date'] = pd.to_datetime(sic_df['date'])

sic_df['year'] = sic_df['date'].dt.year
sic_df['month'] = sic_df['date'].dt.month

sic_df.sort_values('date', inplace=True)
sic_df.reset_index(drop=True, inplace=True)
sic_df = sic_df[['date', 'sic']]
sic_df.set_index('date')
sic_df.index.names = ['Date']

Create_EDA(sic_df)

```

```

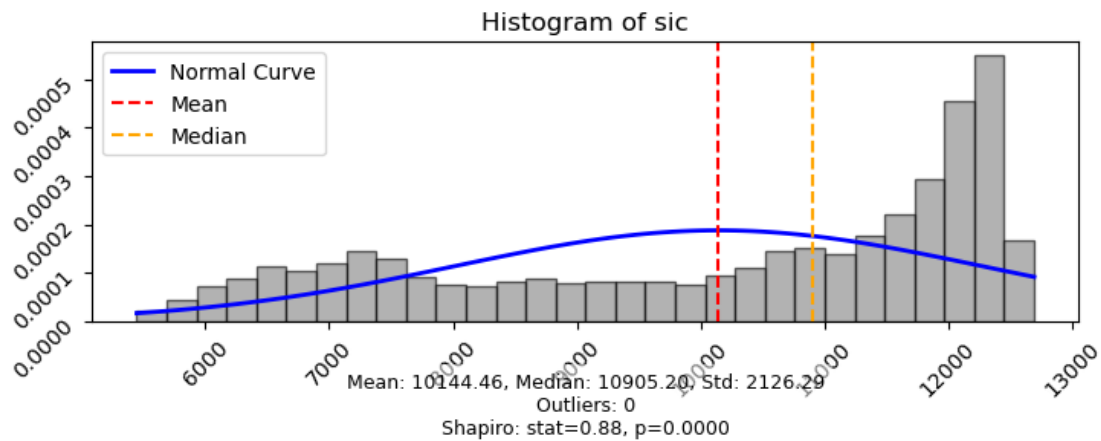
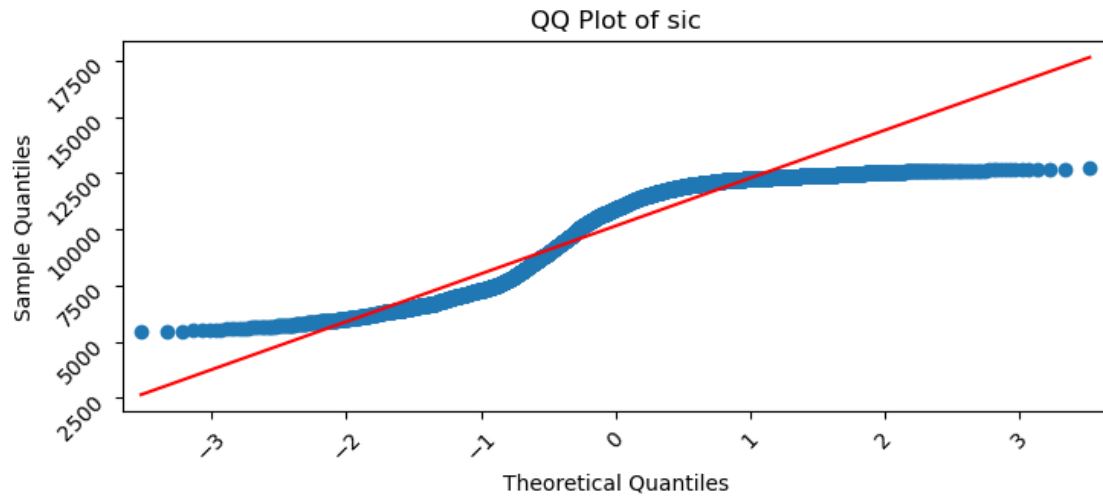
Amount of days with missing values: date    0.0
sic    0.0
dtype: float64

```

Descriptives of labor participation:

	date	sic
count	4748	4748.000000
mean	2014-03-12 02:37:42.510530816	10144.458111
min	2006-01-01 00:00:00	5445.534727
25%	2011-04-02 18:00:00	8208.467553
50%	2014-07-02 12:00:00	10905.204426
75%	2017-10-01 06:00:00	12044.521888
max	2020-12-31 00:00:00	12694.216693
std	NaN	2126.292922

Distribution and QQ plots:



15 Analysis 11 - Load Consumption

```
[46]: import os
import pandas as pd

base_dir = '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/entsoe/
↳csv'
Load_files = os.listdir(base_dir)

dfs = []

for file in Load_files:
    try:
```

```

file_path = os.path.join(base_dir, file)
df = pd.read_csv(file_path)

# Process time column
# Extract start time (first part before ' - ')
df['Start_Time'] = df['Time (CET/CEST)'].str.split(' - ').str[0]

# Convert to datetime
df['datetime'] = pd.to_datetime(
    df['Start_Time'],
    format='%d.%m.%Y %H:%M'
)

# Split into components
df['Day'] = df['datetime'].dt.day
df['Month'] = df['datetime'].dt.month
df['Year'] = df['datetime'].dt.year
df['Hour'] = df['datetime'].dt.hour
df['Minute'] = df['datetime'].dt.minute

# Remove original time columns and temporary columns
df = df.drop(columns=[
    'Time (CET/CEST)',
    'Start_Time',
])

dfs.append(df)

except Exception as e:
    print(f"Skipped {file} due to error: {e}")

# Vertically concatenate all DataFrames
load_df = pd.concat(dfs, ignore_index=True)

# Sort by date descending (newest first)
load_df = load_df.sort_values(
    by=['Year', 'Month', 'Day', 'Hour', 'Minute'],
    ascending=False
).reset_index(drop=True)

load_df = load_df[load_df['Year'] != 2025]
load_df = load_df.drop(columns=['Day-ahead Total Load Forecast [MW] -_
↳Netherlands (NL)'])

# load_df.set_index(["Year", "Month", "Day", "Hour"], inplace=True)

```

```

load_df.rename(columns={"Actual Total Load [MW] - Netherlands (NL)":
↳"LoadConsumption"}, inplace=True)

# import pandas as pd

# Read and preprocess the Excel file
excel_df = pd.read_excel("/Users/ruben/Documents/GitHub/
↳MsCThesisRubenCurriel2024/Data/entsoe/load_2006_2015.xlsx")
excel_nl = excel_df[excel_df['Country'] == 'NL'].drop(columns=['Country'])

# Melt hourly columns and rename to match load_df's target column
excel_melted = excel_nl.melt(
    id_vars=["Year", "Month", "Day", "Coverage ratio"],
    value_vars=list(range(24)), # Columns 0-23 as integers
    var_name="Hour",
    value_name="Actual Total Load [MW] - Netherlands (NL)" # Exact target
↳column name
)
excel_melted["Hour"] = excel_melted["Hour"].astype(int)
excel_melted.rename(columns={"Actual Total Load [MW] - Netherlands (NL)":
↳"LoadConsumption"}, inplace=True)
# excel_melted.set_index(["Year", "Month", "Day", "Hour"], inplace=True)

# Merge with load_df using OUTER join to include all years
merged_df = pd.merge(
    load_df,
    excel_melted,
    on=["Year", "Month", "Day", "Hour"],
    how="outer", # Include all years from both datasets
    suffixes=('', '_excel')
)

# Overwrite the original load column with Excel data where available
merged_df["LoadConsumption"] = merged_df["LoadConsumption_excel"].combine_first(
    merged_df["LoadConsumption"]
)

# Clean up redundant columns
merged_df = merged_df.drop(columns=["LoadConsumption_excel", "Day-ahead Total
↳Load Forecast [MW] - BZN|NL"])

merged_df["Minute"] = merged_df["Minute"].fillna(0).astype(int) # Default to 0
↳if missing

merged_df = merged_df[merged_df.Year != 2025]

```



```
merged_df.head()

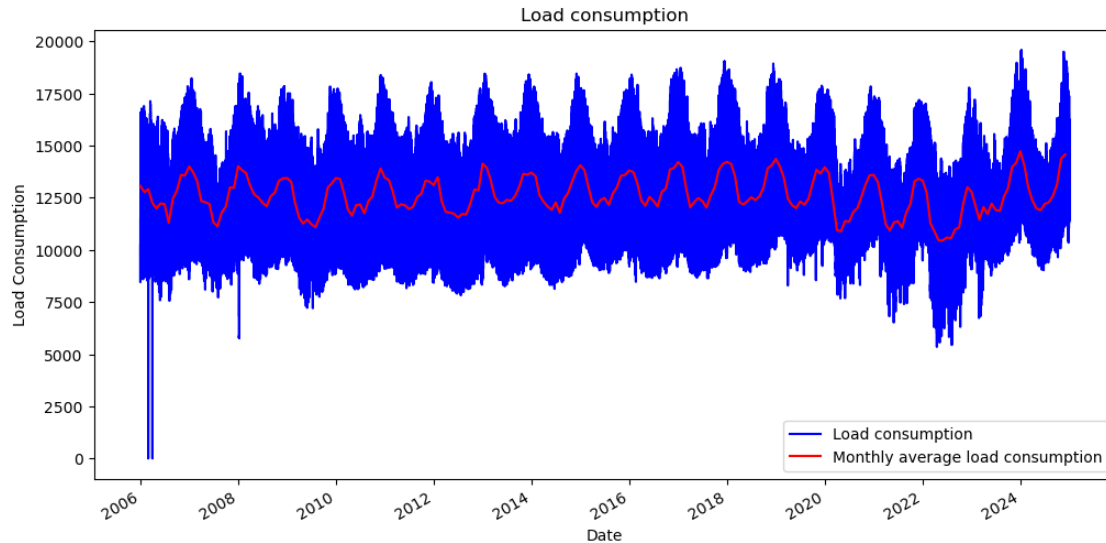
load_NL = merged_df.copy()
load_NL['datetime'] = pd.to_datetime(
    load_NL[['Year', 'Month', 'Day', 'Hour', 'Minute']].rename(columns={
        'Year': 'year',
        'Month': 'month',
        'Day': 'day',
        'Hour': 'hour',
        'Minute': 'minute'
    })
)
```

```
[47]: plt.figure(figsize=(12, 6))
plt.plot(
    load_NL['datetime'],
    load_NL['LoadConsumption'],
    label='Load consumption',
    color='blue'
)

monthly_load = load_NL.groupby(load_NL['datetime'].dt.
    ↪to_period('M'))['LoadConsumption'].mean()

monthly_load.plot(kind='line', color='red', label='Monthly average load_
    ↪consumption')

plt.title('Load consumption')
plt.xlabel('Date')
plt.ylabel('Load Consumption')
plt.legend()
plt.show()
```



15.1 Creating a complete dataset to test

```
[126]: df_LaborParticipation_dt_columns = add_datetime_columns(df_LaborParticipation)
df_Boughthomes_dt_columns = add_datetime_columns(df_Boughthomes)
df_nat_dt_columns = add_datetime_columns(nat_df_filled)
df_ConsumerPriceIndex_dt_columns = add_datetime_columns(df_ConsumerPriceIndex)
df_GdpProductionAndSpending_dt_columns =
    ↪ add_datetime_columns(df_GdpProductionAndSpending)
df_Commodities_dt_columns = add_datetime_columns(df_Commodities)
df_nao_dt_columns = add_datetime_columns(nao)
df_weather_dt_columns = add_datetime_columns(weather)
df_energy_prices_dt_columns = add_datetime_columns(energy_prices)

# Merging Economic Predictors together
CompleteDataset = merge_on_common_time_columns(load_NL,
    ↪ df_LaborParticipation_dt_columns)
CompleteDataset = merge_on_common_time_columns(CompleteDataset,
    ↪ df_Boughthomes_dt_columns)
CompleteDataset = merge_on_common_time_columns(CompleteDataset,
    ↪ df_ConsumerPriceIndex_dt_columns)
CompleteDataset = merge_on_common_time_columns(CompleteDataset,
    ↪ df_GdpProductionAndSpending_dt_columns)
CompleteDataset = merge_on_common_time_columns(CompleteDataset,
    ↪ df_Commodities_dt_columns)
CompleteDataset = merge_on_common_time_columns(CompleteDataset,
    ↪ df_energy_prices_dt_columns)
```

```
# Add climate variables if you want
CompleteDataset = merge_on_common_time_columns(CompleteDataset,
↳df_nat_dt_columns)
CompleDataset = merge_on_common_time_columns(CompleteDataset,
↳df_weather_dt_columns)

CompleteDataset = CompleteDataset.drop(columns=['datetime'])
CompleteDataset
```

```
[126]:      LoadConsumption  Day  Month  Year  Hour  Minute  Coverage ratio \
0          10978.0      1      1  2013      0      0          100.0
1          10978.0      1      1  2013      0      0          100.0
2          10978.0      1      1  2013      0      0          100.0
3          10978.0      1      1  2013      0      0          100.0
4          10978.0      1      1  2013      0      0          100.0
..          ... ..      ...      ...      ...      ...
107         11216.0      1      4  2020      0      0           NaN
108         11574.0      1      7  2020      0      0           NaN
109         11574.0      1      7  2020      0      0           NaN
110         11523.0      1     10  2020      0      0           NaN
111         11523.0      1     10  2020      0      0           NaN
```

```
      Beroepsbevolking|werkloze beroepsbevolking|Werkloze beroepsbevolking \
0          709
1          709
2          709
3          709
4          709
..          ...
107         460
108         528
109         528
110         490
111         490
```

```
      Beroepsbevolking|werkloze beroepsbevolking|Kortdurig \
0          477
1          477
2          477
3          477
4          477
..          ...
107         382
108         428
109         428
110         386
111         386
```

	Beroepsbevolking werkloze beroepsbevolking Langdurig of onbekend	...	\
0	232	...	
1	232	...	
2	232	...	
3	232	...	
4	232	...	
..	...	...	
107	77	...	
108	100	...	
109	100	...	
110	104	...	
111	104	...	

	Natural gas, Europe (\$/mmbtu)	\
0	11.870000	
1	11.870000	
2	11.870000	
3	11.870000	
4	11.870000	
..	...	
107	2.120319	
108	1.801035	
109	1.801035	
110	4.889590	
111	4.889590	

	Liquefied natural gas, Japan (\$/mmbtu)	Natural gas index (2010=100)	\
0	15.890000	107.608179	
1	15.890000	107.608179	
2	15.890000	107.608179	
3	15.890000	107.608179	
4	15.890000	107.608179	
..	...	...	
107	10.009881	37.974040	
108	7.786446	34.908525	
109	7.786446	34.908525	
110	6.179205	54.721155	
111	6.179205	54.721155	

	euro per m3	euro per m3.1	euro per m3.2	euro per kWh	euro per kWh.1	\
0	0.638	0.550	0.282	0.157	0.094	
1	0.638	0.550	0.282	0.157	0.094	
2	0.638	0.550	0.282	0.157	0.094	
3	0.638	0.550	0.282	0.157	0.094	
4	0.638	0.550	0.282	0.157	0.094	
..	...	...	...	...	...	

107	0.809	0.664	0.185	0.121	0.106
108	0.816	0.666	0.182	0.113	0.103
109	0.816	0.666	0.182	0.113	0.103
110	0.816	0.666	0.182	0.113	0.103
111	0.816	0.666	0.182	0.113	0.103

	euro per kWh.2	nat
0	0.073	-0.248083
1	0.073	-0.248083
2	0.073	-0.248083
3	0.073	-0.248083
4	0.073	-0.248083
..	...	...
107	0.068	0.535141
108	0.058	0.577291
109	0.058	0.577291
110	0.058	0.450701
111	0.058	0.450701

[112 rows x 58 columns]

16 Correlation Analysis

```
[136]: from plotly.subplots import make_subplots
import plotly.express as px
import plotly.graph_objects as go
from scipy.stats import pearsonr

# # Calculate correlations with p-values
correlation_results = calculate_correlations_with_pvalues(CompleteDataset)

# # Save to CSV
correlation_results.to_csv(
    '/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/processed_data/
    ↪CompleteCorrelation.csv',
    index=True)
```

```
[137]: correlation_results
```

	Correlation	P_Value
Variable		
LoadConsumption	1.000000	0.000000e+00
Crude oil, WTI (\$/bbl)	-0.498885	2.163029e-08
Crude oil, average (\$/bbl)	-0.477720	1.002451e-07
Crude oil, Brent (\$/bbl)	-0.472100	1.480777e-07
Crude oil, Dubai (\$/bbl)	-0.461585	3.016245e-07

Natural gas, US (\$/mmbtu)	-0.430211	2.198768e-06
Natural gas index (2010=100)	-0.409433	7.375958e-06
euro per m3.2	-0.396018	1.545415e-05
Natural gas, Europe (\$/mmbtu)	-0.382732	3.117103e-05
Opbouw bbp vanuit de finale bestedingen Finale ...	0.378225	3.927961e-05
Year	0.342183	2.217807e-04
Coal, South African ** (\$/mt)	-0.318783	6.133057e-04
CPI 2015 = 100	0.318544	6.194380e-04
Coal, Australian (\$/mt)	-0.308782	9.246019e-04
Gemiddelde verkoopprijs	0.305066	1.073080e-03
Prijsindex verkoopprijzen Prijsindex bestaande ...	0.291296	1.832770e-03
CPI afgeleid 2015 = 100	0.281042	2.685215e-03
Prijsindex verkoopprijzen Ontwikkeling t.o.v. ...	0.270978	3.854112e-03
Opbouw bbp vanuit de finale bestedingen Finale ...	0.254076	6.866908e-03
euro per m3	0.252114	7.325852e-03
Jaarmutatie CPI %	-0.236653	1.199833e-02
euro per m3.1	0.221704	1.880949e-02
Opbouw bbp vanuit de finale bestedingen Finale ...	-0.220151	1.967866e-02
Liquefied natural gas, Japan (\$/mmbtu)	-0.208987	2.700947e-02
Beroepsbevolking werkloze beroepsbevolking Kort...	-0.208052	2.771771e-02
Opbouw bbp vanuit de finale bestedingen Finale ...	-0.206125	2.922649e-02
Opbouw bbp vanuit de finale bestedingen Finale ...	-0.204721	3.036868e-02
Beroepsbevolking Werkloosheidspercentage	-0.196221	3.812396e-02
Opbouw bbp vanuit de finale bestedingen Finale ...	0.194705	3.966856e-02
Beroepsbevolking werkloze beroepsbevolking Werk...	-0.191215	4.342493e-02
Jaarmutatie CPI afgeleid %	-0.189737	4.510170e-02
Opbouw bbp vanuit de finale bestedingen Beschik...	-0.185059	5.077062e-02
Beroepsbevolking werkloze beroepsbevolking Lang...	-0.156295	9.983663e-02
Opbouw bbp vanuit de finale bestedingen Finale ...	0.145892	1.248138e-01
euro per kWh.2	-0.142431	1.341117e-01
Opbouw bbp vanuit de finale bestedingen Beschik...	0.123338	1.951138e-01
Opbouw bbp vanuit de finale bestedingen Beschik...	-0.122292	1.989590e-01
Opbouw bbp vanuit de finale bestedingen Finale ...	-0.104487	2.729110e-01
Opbouw bbp vanuit de finale bestedingen Beschik...	-0.104487	2.729110e-01
euro per kWh.1	0.102668	2.813917e-01
Prijsindex verkoopprijzen Ontwikkeling t.o.v. ...	0.092305	3.330577e-01
Month	-0.089522	3.479004e-01
Totale waarde verkoopprijzen	0.081164	3.949202e-01
Aantal verkochte woningen Aantal verkochte woni...	0.074456	4.352650e-01
Aantal verkochte woningen Ontwikkeling t.o.v. ...	-0.071874	4.513979e-01
Opbouw bbp vanuit de finale bestedingen Beschik...	-0.065818	6.298387e-01
Opbouw bbp vanuit de finale bestedingen Finale ...	0.052529	5.822811e-01
euro per kWh	0.048452	6.119313e-01
Opbouw bbp vanuit de finale bestedingen Finale ...	0.040278	6.732812e-01
Opbouw bbp vanuit de finale bestedingen Finale ...	-0.036788	7.001732e-01
Aantal verkochte woningen Ontwikkeling t.o.v. ...	0.036619	7.014825e-01
Opbouw bbp vanuit de finale bestedingen Finale ...	0.020567	8.295801e-01

Opbouw bbp vanuit de finale bestedingen Beschik...	-0.013637	8.865207e-01
nat	-0.009879	9.176595e-01
Day	NaN	NaN
Hour	NaN	NaN
Minute	NaN	NaN
Coverage ratio	NaN	NaN

```
[138]: import pandas as pd
import numpy as np
import plotly.express as px

# Load precomputed correlations (exclude self-correlation)
corr_df = (
    pd.read_csv('/Users/ruben/Documents/GitHub/MsCThesisRubenCuriel2024/Data/
↳processed_data/CompleteCorrelation.csv')
    .query("Variable != 'LoadConsumption'")
)

# Round for display
corr_df[['Correlation', 'P_Value']] = corr_df[['Correlation', 'P_Value']].
↳round({'Correlation':2, 'P_Value':4})

# Filter significant (p < .05)
sig = corr_df[corr_df['P_Value'] < 0.05]

# Print table of significant correlations
if not sig.empty:
    print("\nSignificant Correlations (p < 0.05):")
    print(sig[['Variable', 'Correlation', 'P_Value']].sort_values('Correlation',
↳ascending=False).to_string(index=False))
else:
    print("\nNo significant correlations found.")

# Top 10 positive & negative
top_pos = sig.nlargest(10, 'Correlation')['Variable'].tolist()
top_neg = sig.nsmallest(10, 'Correlation')['Variable'].tolist()

print("\nTop 10 Positive Correlations:", top_pos)
print("Top 10 Negative Correlations:", top_neg)

# Mark significance for plotting
corr_df['Significance'] = np.where(corr_df['P_Value'] < 0.05, 'Significant',
↳'Not Significant')

# Plotly scatter
fig = px.scatter(
    corr_df,
```

```

x='Correlation',
y='Variable',
color='Significance',
color_discrete_map={'Significant':'green','Not Significant':'red'},
title='Correlation with LoadConsumption ( = 0.05)',
hover_data={'P_Value':True},
category_orders={'Variable': corr_df.sort_values('Correlation')['Variable'].
    ↪tolist()}
)

fig.update_layout(
    xaxis_title='Correlation Coefficient',
    yaxis_title=None,
    xaxis_range=[-1.1, 1.1],
    height=600
)

fig.update_traces(marker=dict(size=10, opacity=0.7),
    hovertemplate="<b>{y}</b><br>Correlation: %{x:.
    ↪2f}<br>P-Value: %{customdata[0]:.4f}<extra></extra>")

fig.show()

```

Significant Correlations (p < 0.05):

Correlation	P_Value	Variable
		Opbouw bbp vanuit de finale bestedingen Finale bestedingen Nationale finale bestedingen Bruto investeringen in vaste activa Overheid
0.38	0.0000	
		Year
0.34	0.0002	
		CPI 2015 = 100
0.32	0.0006	
		Gemiddelde verkoopprijs
0.31	0.0011	
		Prijsindex verkoopprijzen Prijsindex bestaande koopwoningen
0.29	0.0018	
		CPI afgeleid 2015 = 100
0.28	0.0027	
		Prijsindex verkoopprijzen Ontwikkeling t.o.v. een jaar eerder
0.27	0.0039	
		euro per m3
0.25	0.0073	
		Opbouw bbp vanuit de finale bestedingen Finale bestedingen Nationale finale bestedingen Consumptieve bestedingen Overheid
0.25	0.0069	

		euro per m3.1
0.22	0.0188	
		Opbouw bbp vanuit de finale
		bestedingen Finale bestedingen Uitvoer van goederen en diensten Diensten
0.19	0.0397	
		Beroepsbevolking werkloze beroepsbevolking Werkloze beroepsbevolking
-0.19	0.0434	
		Jaarmutatie CPI afgeleid %
-0.19	0.0451	
		Beroepsbevolking Werkloosheidspercentage
-0.20	0.0381	
		Opbouw bbp vanuit de finale bestedingen Finale
		bestedingen Nationale finale bestedingen Bruto investeringen in vaste
		activa Totaal
	-0.20	0.0304
		Liquefied natural gas, Japan (\$/mmbtu)
-0.21	0.0270	
		Opbouw bbp vanuit de finale bestedingen Finale bestedingen Nationale finale
		bestedingen Bruto investeringen in vaste activa Bedrijven en huishoudens
-0.21	0.0292	
		Beroepsbevolking werkloze beroepsbevolking Kortdurig
-0.21	0.0277	
		Opbouw bbp vanuit de
		finale bestedingen Finale bestedingen Nationale finale bestedingen Totaal
-0.22	0.0197	
		Jaarmutatie CPI %
-0.24	0.0120	
		Coal, Australian (\$/mt)
-0.31	0.0009	
		Coal, South African ** (\$/mt)
-0.32	0.0006	
		Natural gas, Europe (\$/mmbtu)
-0.38	0.0000	
		euro per m3.2
-0.40	0.0000	
		Natural gas index (2010=100)
-0.41	0.0000	
		Natural gas, US (\$/mmbtu)
-0.43	0.0000	
		Crude oil, Dubai (\$/bbl)
-0.46	0.0000	
		Crude oil, Brent (\$/bbl)
-0.47	0.0000	
		Crude oil, average (\$/bbl)
-0.48	0.0000	
		Crude oil, WTI (\$/bbl)
-0.50	0.0000	

Top 10 Positive Correlations: ['Opbouw bbp vanuit de finale bestedingen|Finale

```

bestedingen|Nationale finale bestedingen|Bruto investeringen in vaste
activa|Overheid', 'Year', 'CPI | 2015 = 100', 'Gemiddelde verkoopprijs',
'Prijsindex verkoopprijzen|Prijsindex bestaande koopwoningen', 'CPI afgeleid |
2015 = 100', 'Prijsindex verkoopprijzen|Ontwikkeling t.o.v. een jaar eerder',
'Opbouw bbp vanuit de finale bestedingen|Finale bestedingen|Nationale finale
bestedingen|Consumptieve bestedingen|Overheid', 'euro per m3', 'euro per m3.1']
Top 10 Negative Correlations: ['Crude oil, WTI | ($/bbl)', 'Crude oil, average |
($/bbl)', 'Crude oil, Brent | ($/bbl)', 'Crude oil, Dubai | ($/bbl)', 'Natural
gas, US | ($/mmbtu)', 'Natural gas index | (2010=100)', 'euro per m3.2',
'Natural gas, Europe | ($/mmbtu)', 'Coal, South African ** | ($/mt)', 'Coal,
Australian | ($/mt)']

```

```

[139]: df_prophet = CompleteDataset
df_prophet = CompleteDataset.rename(columns={"DateUTC": "ds", "LoadConsumption":
↪ "y"})

df_prophet['ds'] = pd.to_datetime(
    df_prophet[['Year', 'Month', 'Day', 'Hour', 'Minute']]
)

train_size = int(0.8 * len(df_prophet))
train_data, test_data = df_prophet[:train_size], df_prophet[train_size:]

df_prophet.head()

```

```

[139]:
      y  Day  Month  Year  Hour  Minute  Coverage ratio \
0  10978.0    1     1  2013     0         0          100.0
1  10978.0    1     1  2013     0         0          100.0
2  10978.0    1     1  2013     0         0          100.0
3  10978.0    1     1  2013     0         0          100.0
4  10978.0    1     1  2013     0         0          100.0

```

```

      Beroepsbevolking|werkloze beroepsbevolking|Werkloze beroepsbevolking \
0                                709
1                                709
2                                709
3                                709
4                                709

```

```

      Beroepsbevolking|werkloze beroepsbevolking|Kortdurig \
0                                477
1                                477
2                                477
3                                477
4                                477

```

	Beroepsbevolking werkloze beroepsbevolking Langdurig of onbekend	...	\
0	232	...	
1	232	...	
2	232	...	
3	232	...	
4	232	...	

	Liquefied natural gas, Japan (\$/mmbtu)	Natural gas index (2010=100)	\
0	15.89	107.608179	
1	15.89	107.608179	
2	15.89	107.608179	
3	15.89	107.608179	
4	15.89	107.608179	

	euro per m3	euro per m3.1	euro per m3.2	euro per kWh	euro per kWh.1	\
0	0.638	0.55	0.282	0.157	0.094	
1	0.638	0.55	0.282	0.157	0.094	
2	0.638	0.55	0.282	0.157	0.094	
3	0.638	0.55	0.282	0.157	0.094	
4	0.638	0.55	0.282	0.157	0.094	

	euro per kWh.2	nat	ds
0	0.073	-0.248083	2013-01-01
1	0.073	-0.248083	2013-01-01
2	0.073	-0.248083	2013-01-01
3	0.073	-0.248083	2013-01-01
4	0.073	-0.248083	2013-01-01

[5 rows x 59 columns]

```
[140]: def initialize_prophet(regressors, add_seasonality, fourier_weekly,
    ↪fourier_annual):
    """
    Initialize prophet. Either with specified seasonalities or default.
    Add regressors if any are entered.
    """
    if add_seasonality:
        model = Prophet(
            daily_seasonality=False,
            weekly_seasonality=fourier_weekly,
            yearly_seasonality=fourier_annual,
        )
    else:
        model = Prophet()
```

```

    if len(regressors) > 0:
        for regressor in regressors:
            model.add_regressor(regressor)

    return model

def run_model(train, test, regressors, add_seasonality, fourier_weekly,
    ↪fourier_annual):
    """
    Run model.
    Give predictions.
    Calculate RMSE.
    """

    # initialize model
    model = initialize_prophet(regressors, add_seasonality, fourier_weekly,
    ↪fourier_annual)

    # fit model
    model.fit(train)

    # predict with model
    forecast = model.predict(test)
    forecast['yhat_rounded'] = forecast['yhat'].apply(lambda x: math.floor(x))

    # calculate rmse
    rmse = sqrt(mean_squared_error(test['y'],
    ↪forecast['yhat_rounded'][-len(test):]))

    corr, p_value = pearsonr(y_test, predictions)
    print(f'Correlation: {corr} and p-value: {p_value} on Test Data')
    return rmse

```

```

[141]: # initialize model
model = initialize_prophet(regressors=[], add_seasonality=False,
    ↪fourier_weekly=0, fourier_annual=0)

# fit model
model.fit(train_data)

# predict with model
forecast = model.predict(test_data)
forecast['yhat_rounded'] = forecast['yhat'].apply(lambda x: math.floor(x))

# calculate rmse
print("RMSE:", sqrt(mean_squared_error(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])))

```

```

corr, p_value = pearsonr(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])
print(f'Correlation: {corr} and p-value: {p_value} on Test Data')

# plot
fig1 = model.plot(forecast)

fig2 = model.plot_components(forecast)

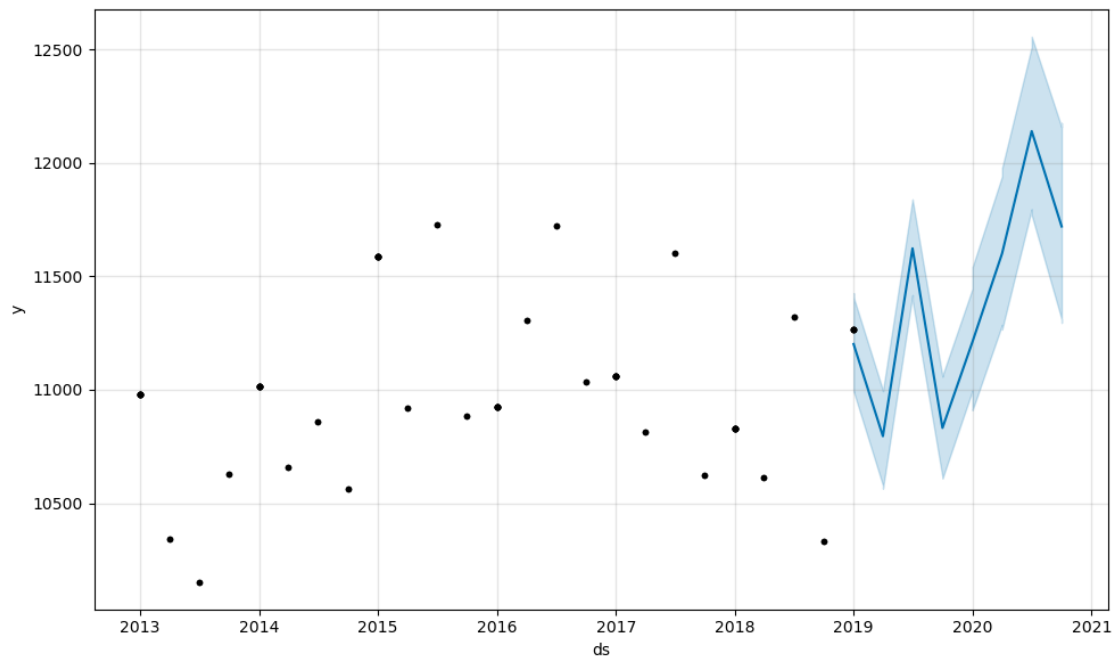
```

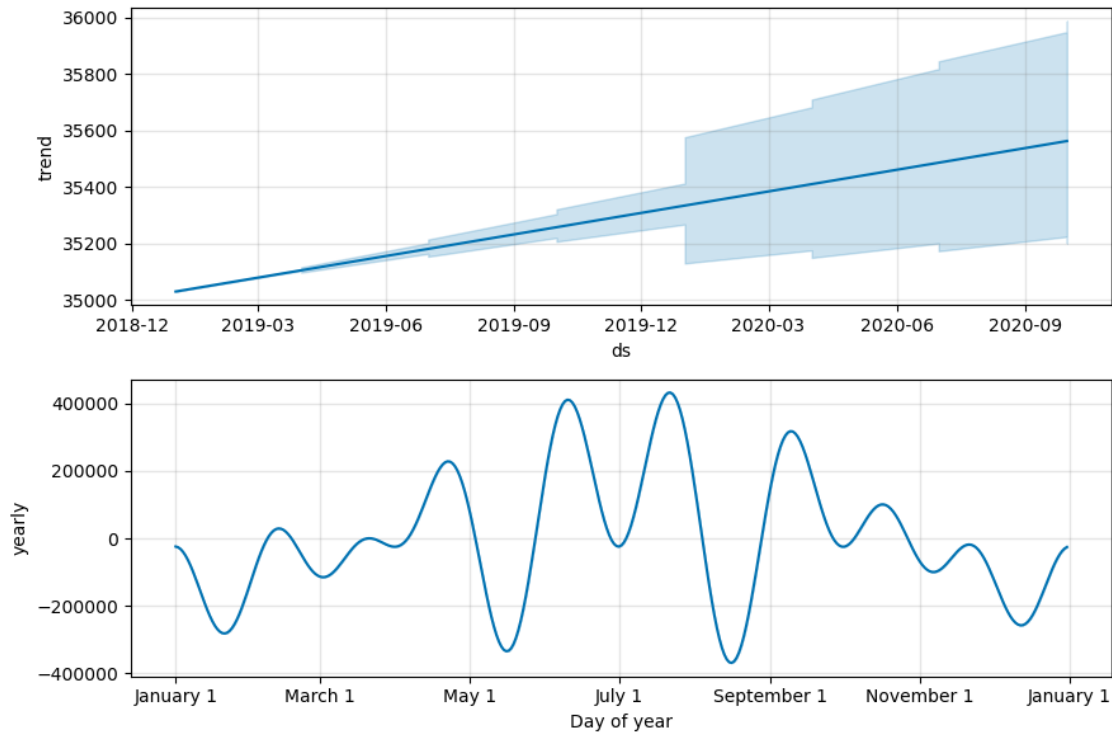
13:42:11 - cmdstanpy - INFO - Chain [1] start processing

13:42:11 - cmdstanpy - INFO - Chain [1] done processing

RMSE: 322.7673142225445

Correlation: 0.49383597425322473 and p-value: 0.01662374261421361 on Test Data





```
[142]: # initialize model
model = initialize_prophet(
    regressors=top_neg[:3],
    add_seasonality=True,
    fourier_weekly=7,
    fourier_annual=7
)

# fit model
model.fit(train_data)

# predict with model
forecast = model.predict(test_data)
forecast['yhat_rounded'] = forecast['yhat'].apply(lambda x: math.floor(x))

# calculate rmse
print("RMSE:", sqrt(mean_squared_error(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])))
corr, p_value = pearsonr(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])
print(f'Correlation: {corr} and p-value: {p_value} on Test Data')

# plot
```

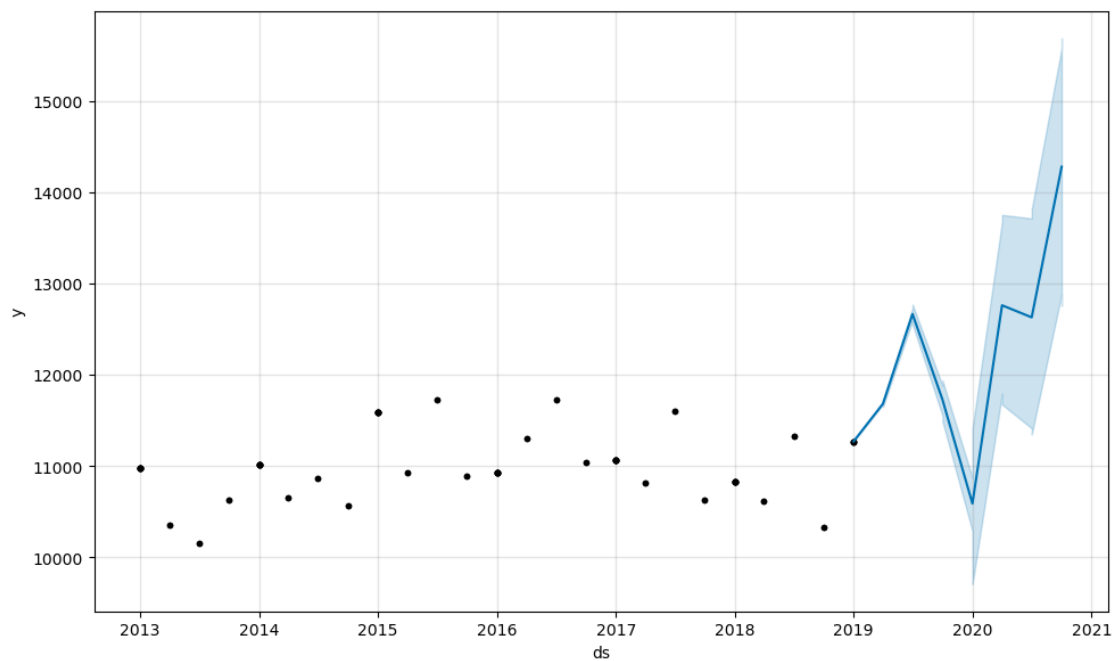
```
fig1 = model.plot(forecast)
fig2 = model.plot_components(forecast)
```

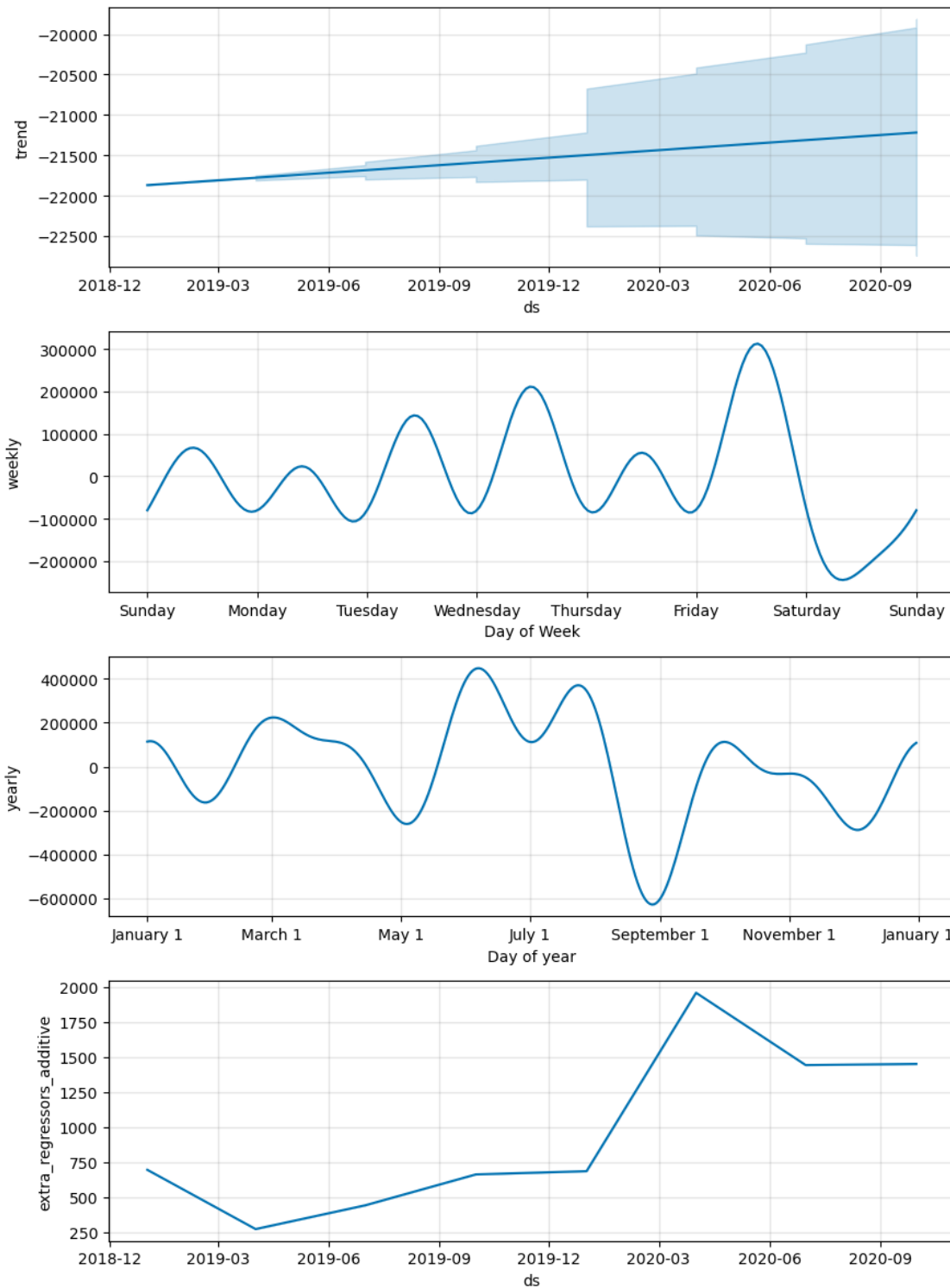
13:42:18 - cmdstanpy - INFO - Chain [1] start processing

13:42:39 - cmdstanpy - INFO - Chain [1] done processing

RMSE: 1102.2625545351864

Correlation: 0.6982098326711429 and p-value: 0.00021153281026008432 on Test Data





```
[143]: # initialize model
model = initialize_prophet(
```



```

    regressors=top_pos[:3],
    add_seasonality=True,
    fourier_weekly=7,
    fourier_annual=7
)

# fit model
model.fit(train_data)

# predict with model
forecast = model.predict(test_data)
forecast['yhat_rounded'] = forecast['yhat'].apply(lambda x: math.floor(x))

# calculate rmse
print("RMSE:", sqrt(mean_squared_error(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])))
corr, p_value = pearsonr(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])
print(f'Correlation: {corr} and p-value: {p_value} on Test Data')

# plot
fig1 = model.plot(forecast)
fig2 = model.plot_components(forecast)

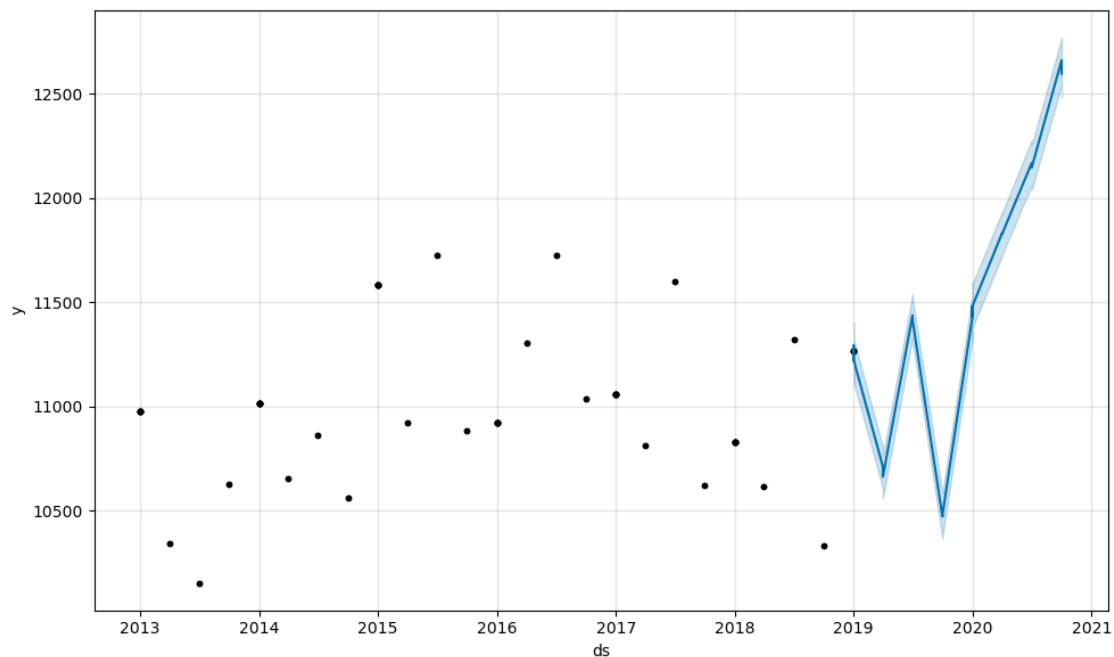
```

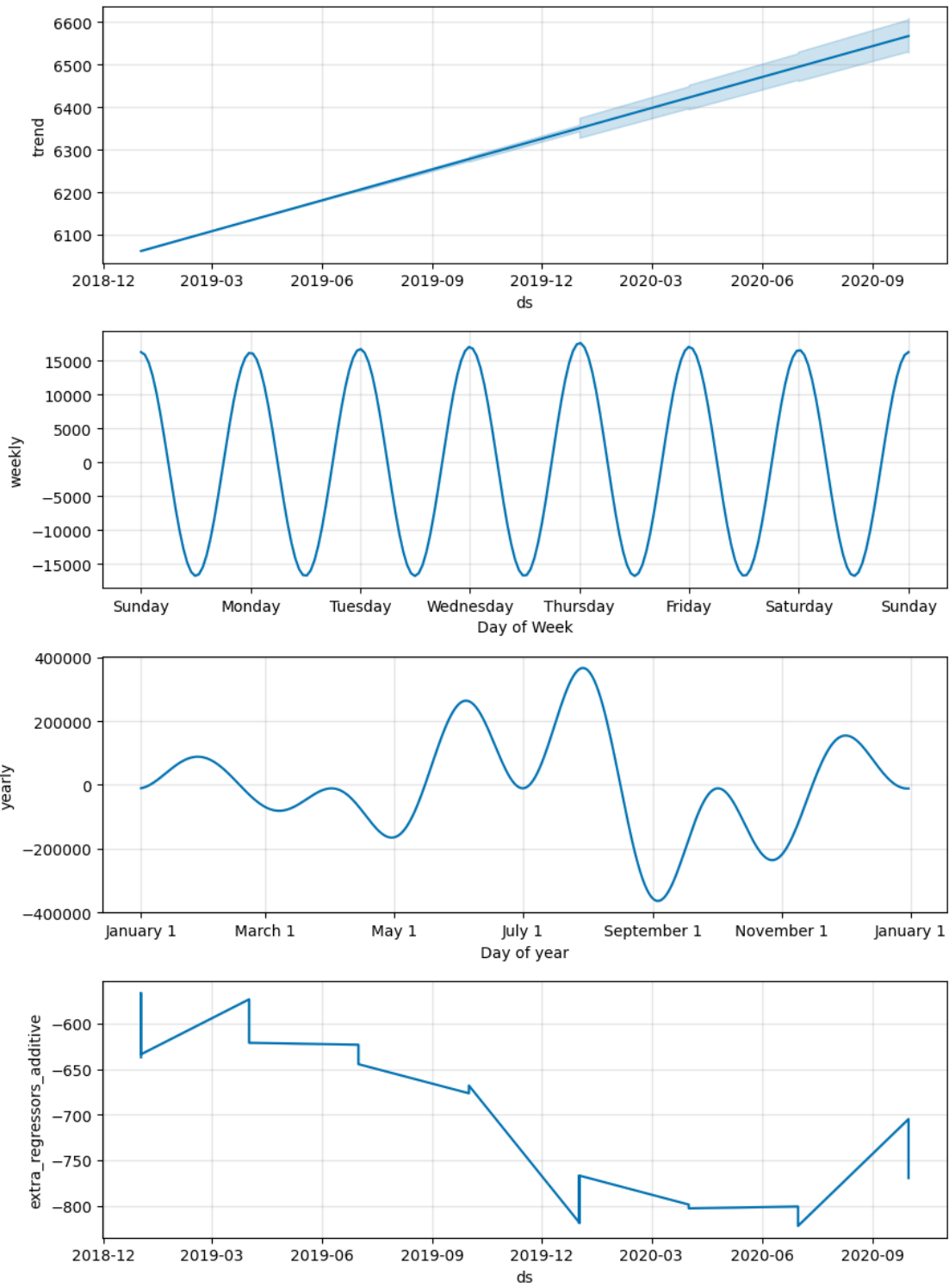
13:42:54 - cmdstanpy - INFO - Chain [1] start processing

13:42:54 - cmdstanpy - INFO - Chain [1] done processing

RMSE: 565.5364901923929

Correlation: 0.23279907999156152 and p-value: 0.28507955414061303 on Test Data





```
[144]: # initialize model
model = initialize_prophet(
    regressors=top_neg[:3]+top_pos[:2],
    add_seasonality=True,
    fourier_weekly=7,
    fourier_annual=7
)

# fit model
model.fit(train_data)

# predict with model
forecast = model.predict(test_data)
forecast['yhat_rounded'] = forecast['yhat'].apply(lambda x: math.floor(x))

# calculate rmse
print("RMSE:", sqrt(mean_squared_error(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])))
corr, p_value = pearsonr(test_data['y'],
    ↪forecast['yhat_rounded'][-len(test_data):])
print(f'Correlation: {corr} and p-value: {p_value} on Test Data')

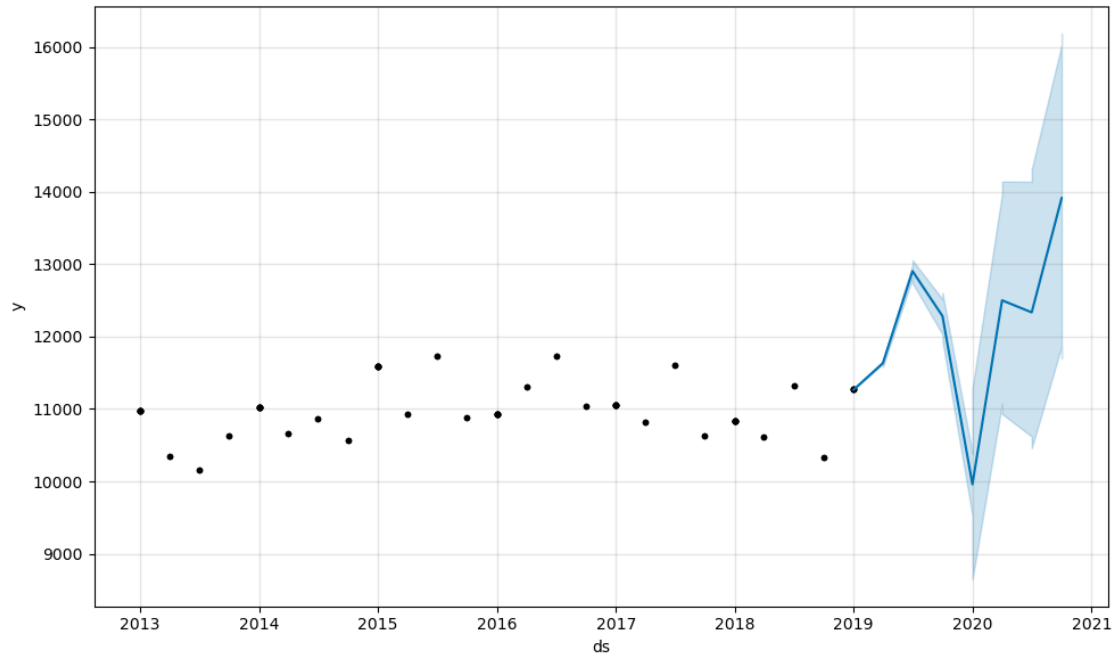
# plot
fig1 = model.plot(forecast)
fig2 = model.plot_components(forecast)
```

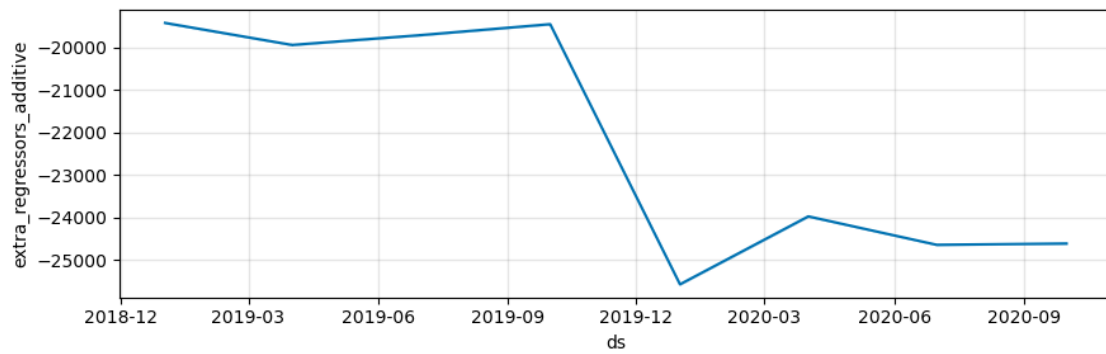
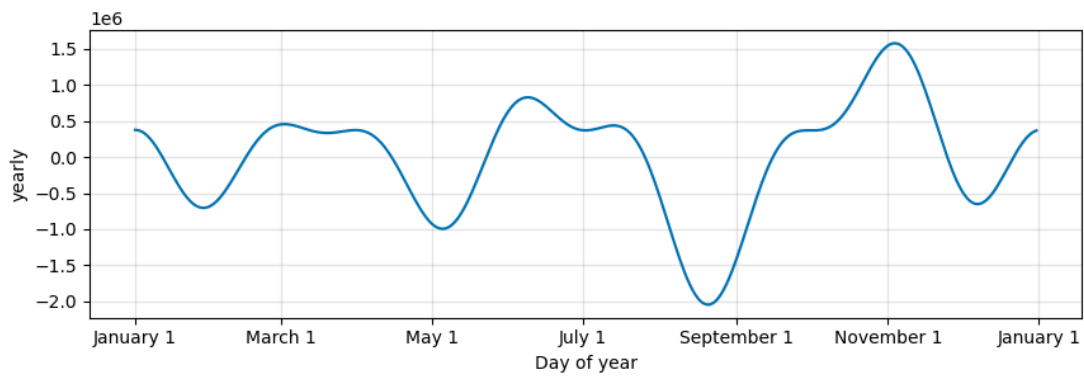
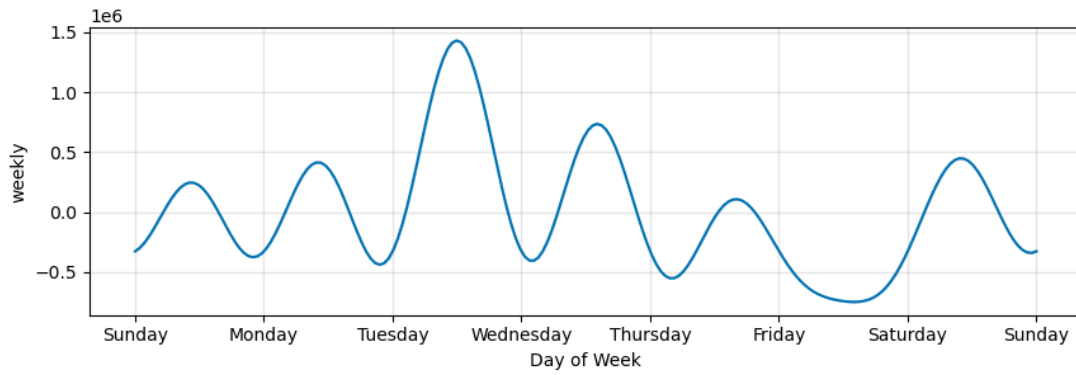
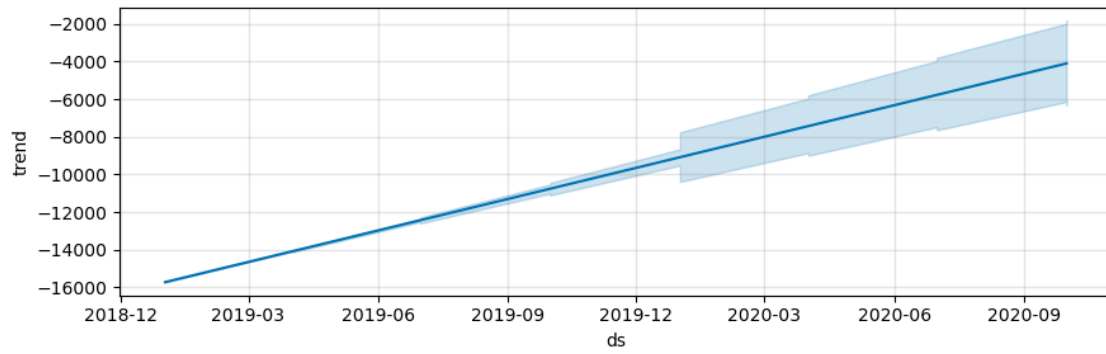
13:43:01 - cmdstanpy - INFO - Chain [1] start processing

13:43:24 - cmdstanpy - INFO - Chain [1] done processing

RMSE: 1195.7753169787593

Correlation: 0.7763323864332282 and p-value: 1.332685357230854e-05 on Test Data





[]: